A Project Report on

NLP BASED RESUME ANALYSIS AND RANKING SYSTEM

submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF ENGINEERING

By

Aabha Bhide: 22106093

Aditi Gadhave: 22106079

Niharika Bandekar: 22106136

Sumeet Gupta: 22106071

Under the Guidance of

Prof. Vijaya Bharathi Jagan

Department of Computer Science & Engineering (Artificial Intelligence & Machine Learning)

A.P. SHAH INSTITUTE OF TECHNOLOGY, THANE

UNIVERSITY OF MUMBAI

2025-2026

A. P. SHAH INSTITUTE OF TECHNOLOGY, THANE

CERTIFICATE

This is to certify that the project entitled "NLP Based Resume Analysis and Ranking System" is a bonafide work of **Aabha Bhide(22106093), Aditi Gadhave(22106079), Niharika Bandekar(22106136), Sumeet Gupta(22106071)** submitted to the University of Mumbai in partial fulfilment of the requirement for the award of the degree of **Bachelor of Engineering** in **Computer Science & Engineering (Artificial Intelligence & Machine Learning**).

Prof. Vijaya Bharathi Jagan
Guide

Prof. Sayali P. Badhan
Project Co-ordinator

Dr. Jaya Gupta
Head of Department

Dr.Uttam D. Kolekar
Principal

A. P. SHAH INSTITUTE OF TECHNOLOGY, THANE

Project Report Approval for B.E.

This project report entitled "***NLP Based Resume Analysis and Ranking System***" by ***Aabha Bhide, Aditi Gadhave, Niharika Bandekar, Sumeet*** **Gupta** is approved for the degree of ***Bachelor of Engineering*** in ***Computer Science & Engineering (Artificial Intelligence & Machine Learning)***, ***2025-26***.

Examiner Name	Signature
1._____	_____
2._____	_____

Date:

Place:

Declaration

We declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Aabha Bhide (22106093)

Aditi Gadhave (22106079)

Niharika Bandekar (22106136)

Sumeet Gupta (22106071)

Date:

Abstract

Manual resume screening and keyword-based applicant tracking systems fail to capture semantic relevance, provide actionable feedback and support interpretable candidate ranking. This project presents an NLP-based resume analysis and ranking system designed for student self-assessment and recruiter decision support. The system computes a reproducible weighted Applicant Tracking System (ATS) score through five integrated components. Term Frequency–Inverse Document Frequency (TF-IDF) cosine similarity quantifies lexical overlap between resumes and job descriptions. Sentence-transformer–based experience matching captures semantic relationships between employment histories and position requirements beyond surface keywords. Rule-based formatting evaluation assesses document structure and presentation quality against professional standards. Explicit skill overlap analysis identifies direct correspondences between candidate competencies and technical requirements. Education alignment checks that the candidate's qualifications meet the job requirements. These components aggregate through calibrated weighting to produce the final ATS score. A hybrid symbolic–semantic skill gap module extends job skills using an ontology to identify related and prerequisite competencies, then applies zero-shot natural language inference to estimate confidence scores for missing skills based on contextual resume evidence. For recruiter decision support, candidate ranking combines structured features with semantic similarity through a lightweight neural ranking model. SHapley Additive exPlanations (SHAP)-based post-hoc explainability decomposes predictions for top-k candidates, enabling recruiters to understand specific ranking factors and facilitating transparent, defensible selection decisions. Implemented using FastAPI, Facebook AI Similarity Search (FAISS) and SQLite, the core ATS and ranking modules run locally with reproducible outcomes, eliminating external cloud dependencies and ensuring consistent scoring across deployment contexts. The system supports Sustainable Development Goal 4 (Quality Education) by providing students with objective feedback for informed skill development and Sustainable Development Goal 8 (Decent Work and Economic Growth) through interpretable, reproducible hiring processes that emphasize transparent evaluation criteria and equitable candidate assessment.

Keywords: Applicant Tracking System(ATS), Term Frequency–Inverse Document Frequency (TF-IDF) ,Sentence Transformers, Zero-Shot Learning Natural Language Inference, SHapley Additive exPlanations (SHAP), Facebook AI Similarity Search (FAISS), Skills Ontology

Table of Contents

1. Introduction 1

2. Literature Survey 3

2.1 History 3

2.2 Literature Survey 4

3. Limitation of Existing System 11

4. Problem Statement and Objectives 13

4.1 Problem Statement 13

4.2 Objectives 14

4.3 Scope 16

5. Proposed System 19

5.1 System Overview 20

5.2 Student Modules 21

5.2.1 Job Browsing & Application Module: 21

5.2.2 Resume Ingestion and Parsing Module 22

5.2.3 Job-Match Analysis Module 22

5.2.4 Explainability and Feedback Layer 25

5.2.5 Skill Gap Module 25

5.2.6 Mock Interview Module 27

5.2.7 Application Tracking & Progress Monitoring Module 27

5.3 HR modules 28

5.3.1 Job Posting Management Module: 28

5.3.2 Application & Interview Management Module 28

5.3.3 Bulk Resume Upload Module 29

5.3.4 Candidate Ranking & Explainability Module 29

6. Experimental Setup 32

6.1 Details of Database and Input to System 32

6.2 Performance Evaluation Parameters 33
6.3 Software and Hardware Setup 33
7. Results and Discussion 36
7.1 Interpretation of Results 36
7.2 Strengths of the Proposed System 37
7.3 Limitations and Constraints 37
7.4 Results and System Outputs 37
7.5 Practical Implications 39
7.6 Technical Trade-offs 39
7.7 Implementation 39
8. Conclusion and Future Work 49
8.1 Conclusion 49
8.2 Future Work 50
References 51
Publication 53

LIST OF FIGURES

Figure 1: System Context Diagram 20
Figure 2: Functional Architecture Diagram 21
Figure 3: Landing Page 40
Figure 4: Register and Sign in Page 40
Figure 5: Student Dashboard 41
Figure 6: Student Job Browsing 41
Figure 7: Student Resume & Job Matcher 42
Figure 8: Student Resume Analysis 42
Figure 9: Student: Recommendation Courses & Mock Interviews 43
Figure 10: Job Matching Analysis 43
Figure 11: Student Resume Analysis (Overview & Strengths) 44
Figure 12: Student Resume Analysis (Improvements & Skills) 44
Figure 13: Student Profile Updation 45
Figure 14: Student Activity History 45
Figure 15: HR Dashboard 46
Figure 16: HR Job Posting 46
Figure 17: HR Application Manager 47
Figure 18: HR Bulk Resume Upload 47
Figure 19: HR Candidate Ranking 48
Figure 20: HR Candidate Ranking Analysis 48

LIST OF TABLES

Table 1: Summary of Literature Survey 8

ABBREVAION

AI	*Artificial Intelligence*
ML	*Machine Learning*
NLP	*Natural Language Processing*
ATS	*Applicant Tracking System*
TF-IDF	*Term Frequency–Inverse Document Frequency*
NLI	*Natural Language Inference*
NER	*Named Entity Recognition*
LDA	*Latent Dirichlet Allocation*
FAISS	*Facebook AI Similarity Search*
API	*Application Programming Interface*
REST	*Representational State Transfer*
JWT	*JSON Web Token*
FSM	*Finite State Machine*
UUID	*Universally Unique Identifier*
SQL	*Structured Query Language*
ACID	*Atomicity, Consistency, Isolation, Durability*
PDF	*Portable Document Format*
DOCX	*Microsoft Word Document Format*
UI	*User Interface*
UX	*User Experience*
ReLU	*Rectified Linear Unit*
SHAP	*SHapley Additive exPlanations*

Chapter 1

Introduction

1. Introduction

In the contemporary job market, organizations face challenges in managing large volumes of job applications, with popular roles attracting hundreds or thousands of resumes in a short time [1]. Manual screening is time-consuming, error-prone and inconsistent, leading to delays, bias and the risk of overlooking qualified candidates [5]. Traditional Applicant Tracking Systems attempt to address this but rely heavily on keyword matching, which fails to capture semantic meaning and contextual relevance [2], [8]. As a result, candidates may be incorrectly filtered based on terminology differences or keyword overuse, creating a gap between job requirements and actual suitability [3], [9]. This highlights the need for more intelligent resume evaluation methods [5].

To address these limitations, an NLP-based Resume Analysis and Ranking System is proposed, leveraging Natural Language Processing, machine learning and deep learning techniques [2]. The system combines TF-IDF with cosine similarity for keyword relevance [15], Sentence Transformer (MiniLM) embeddings with FAISS for semantic matching [3] and JobBERT for skill extraction from unstructured text [10], [14]. Additionally, BART-based Natural Language Inference is used for skill-entailment analysis, ensuring that candidate experience is

meaningfully aligned with required skills. These techniques together provide transparent scoring and detailed skill-gap insights.

The system is implemented using a FastAPI backend, React with Chakra UI frontend and SQLite database [6], ensuring efficient processing and usability. Recruiters receive ranked candidate lists with explainable criteria [8], while candidates gain insights into their strengths and areas for improvement. Overall, the system improves hiring efficiency, enhances transparency and supports accurate, consistent candidate selection, offering a comprehensive solution to modern recruitment challenges.

Chapter 2

Literature Survey

2. Literature Survey

2.1 History

The domain of recruitment automation and resume screening has evolved significantly, moving from manual evaluation to intelligent, AI-driven systems aimed at improving efficiency, accuracy, scalability and fairness. In the early stages (1980s–early 2000s), recruiters manually reviewed resumes, relying on subjective judgment. Although this allowed contextual understanding, it was time-consuming, inconsistent and prone to bias, making it unsuitable as application volumes increased.

The emergence of online job portals such as Monster and Naukri in the early 2000s led to the development of Applicant Tracking Systems (ATS), which used rule-based keyword matching to filter resumes. While these systems reduced manual effort, they lacked semantic understanding and often resulted in inaccurate filtering. In the 2010s, Machine Learning techniques like TF-IDF and cosine similarity improved textual relevance measurement, but they still relied on statistical representations and could not fully capture contextual meaning.

Advancements in Natural Language Processing (NLP) introduced more effective solutions for handling unstructured resume data. Resume parsing techniques enabled extraction of structured information such as skills, education and experience [2], while transformer-based models like Sentence-BERT improved semantic similarity and contextual matching between resumes and job descriptions [3]. Additional research focused on anomaly detection to identify fraudulent content [4], explainable AI to reduce bias and improve transparency [5] and security mechanisms to protect sensitive recruitment data [6]. Recruitment analytics and visualization tools further supported data-driven decision-making.

Recent approaches emphasize large-scale candidate retrieval and ranking using hybrid models that combine rule-based, statistical and deep learning methods [1]. However, most existing systems address individual components such as parsing, matching or ranking in isolation. The proposed NLP-Based Resume Analysis and Ranking System addresses this gap by integrating TF-IDF, transformer-based semantic models, ontology-driven skill mapping, natural language inference and explainable ranking within a unified, end-to-end framework, ensuring both accuracy and interpretability in recruitment workflows.

2.2 Literature Survey

[1] Machine Learning-Based Human Resource Management Information Retrieval and Classification Algorithm

Year and Authors: 2024, Wen Li and XiuKao Zhou

This paper applies machine learning techniques for information retrieval and document classification to streamline HR data management. It improves the efficiency of candidate search and filtering using ranking-based retrieval models, a concept reflected in this project's FAISS-powered semantic search engine for job–resume matching.

[2] Automated Resume Parsing: A Natural Language Processing Approach

Year and Authors: 2023, Sougandh, Thatavarthi Giri, Nithish Sagar Reddy and Meena Belwal

This paper focuses on automating the extraction of structured information such as skills, education and experience from unstructured resume documents. It employs NLP libraries like spaCy and Transformer models to identify key sections and entities. The work highlights how modern parsing systems can replace manual extraction by leveraging language models for entity recognition and document segmentation.

[3] VacancySBERT: Representation of Titles and Skills for Semantic Similarity Search

Year and Authors: 2023, Maiia Bocharova, Eugene Malakhov and Vitaliy Mezhuyev

This research proposes Sentence-BERT (SBERT) as a method for embedding job titles and skills to measure semantic similarity between resumes and job descriptions. The model ensures that candidate–job matching goes beyond keyword overlap by capturing contextual meaning. This aligns closely with the proposed system's semantic similarity component based on Sentence Transformers (MiniLM).

[4] A Machine Learning Approach to Detecting Fraudulent Job Types

Year and Authors: 2023, Marcel Naudé, Kolawole John Adebayo and Rohan Nanda

This work addresses the issue of fraudulent or exaggerated resume content by employing AI-based anomaly detection. Using outlier detection and data consistency analysis, it identifies suspicious applications or false information. This contributes to secure and reliable recruitment automation, complementing the proposed system's validation and verification logic.

[5] Enhancing Diversity and Reducing Bias in Recruitment through AI: A Review of Strategies and Challenges

Year and Authors: 2023, Ramakrishnan Vivek

This review discusses how AI and explainable machine learning can help minimize unconscious bias in recruitment. It emphasizes fairness, transparency and interpretability as core values of ethical HR technology. These principles directly influence the design philosophy of the proposed NLP-based ranking system, which ensures explainable and equitable candidate assessment.

[6] Secured Recruitment Management System with Data Retention Approach

Year and Authors: 2023, Mashitah Mohd Tobroni and Isredza Rahmi A. Hamid

This study focuses on implementing data security in recruitment systems, emphasizing JWT authentication, role-based access control, encryption and CSRF protection. The proposed system adopts a similar security model through FastAPI's JWT and role-based authentication to ensure privacy of sensitive candidate and recruiter information.

[7] Illuminating HR Insights: Unraveling Patterns through Data Analytics and Visualization

Year and Authors: 2023, Akshata Upadhye

This paper explores data analytics and visualization techniques to provide actionable insights for HR professionals. It introduces dashboards for analyzing applicant trends, skill gaps and hiring patterns. The proposed system extends this concept by integrating interactive analytics dashboards (using Recharts and Chakra UI) for both students and HR users to monitor performance and recruitment metrics.

[8] Job Candidate Rank Approach Using Machine Learning Techniques

Year and Authors: 2021, Lamiaa Mostafa and Sara Beshir

This paper introduces machine learning models for candidate scoring and ranking based on resume features. It applies methods like vector similarity, regression and classification to generate unbiased, data-driven rankings. The study establishes the foundation for systems that use ML-based ranking pipelines similar to the proposed project's RankNet-style candidate evaluation mechanism.

[12] CV Recommendation Model Using Latent Dirichlet Allocation (LDA)

Year and Authors: 2024, J. Tan

This paper presents a CV recommendation model based on Latent Dirichlet Allocation (LDA), a probabilistic topic modeling technique used to identify hidden thematic structures within textual data. The approach models resumes and job descriptions as distributions over latent topics, enabling the system to recommend candidates based on topic similarity rather than direct keyword overlap. This enhances the ability to capture broader contextual relevance across candidate profiles. However, LDA-based models are limited in capturing fine-grained semantic relationships and contextual nuances compared to transformer-based architectures. The proposed system builds upon this concept by incorporating advanced sentence-transformer embeddings and semantic similarity techniques, thereby achieving more precise and context-aware candidate-job matching while retaining interpretability.

[13] How AI-Generated Resumes Impact the Staffing Industry

Year and Authors: 2025, Recruitics Research

This paper explores the increasing impact of AI-generated resumes on modern recruitment processes. It highlights how candidates use AI tools to enhance formatting, keyword usage and content tailoring, which can affect the reliability of traditional screening systems. The study raises concerns about authenticity, over-optimization and the challenge of distinguishing genuine skills from artificially enhanced profiles. To address these issues, the proposed system incorporates semantic analysis, rule-based validation and explainable scoring, ensuring transparent, interpretable and more reliable candidate evaluation despite AI-driven resume optimization.

The reviewed research papers collectively highlight major advancements in NLP and ML for recruitment automation. Their key aspects including domain, algorithm, dataset and identified research gaps are summarized in Table 1 mentioned below.

Table 1: Summary of Literature Survey

Year	Title	Domain	Algorithm	Dataset	Gap
2024	Machine Learning-Based HR Management Information Retrieval and Classification Algorithm	HR Data Retrieval & Classification	Information retrieval (IR) models, ML classifiers, ranking-based retrieval	HR document repository	Emphasizes document retrieval; does not perform ATS or candidate–job matching.
2023	Automated Resume Parsing: A Natural Language Processing Approach	Resume Parsing, Information Extraction	spaCy NER, PyMuPDF for text extraction, Transformer models for entity recognition	Custom dataset of PDF/DOCX resumes	Focuses only on parsing structured info (skills, education, experience); lacks candidate ranking or ATS scoring.
2023	VacancySBERT: Representation of Titles and Skills for Semantic Similarity Search	Semantic Matching in Recruitment	Sentence-BERT (SBERT) for embeddings, cosine similarity for semantic search	Proprietary job–resume corpus	Addresses semantic similarity but does not integrate ranking or skill gap detection.

2023	A Machine Learning Approach to Detecting Fraudulent Job Types	Resume Verification, Fraud Detection	Outlier detection, anomaly detection, AI-based consistency checks	Online job postings dataset	Detects fraud but not suitable for resume–job semantic analysis or scoring.
2023	Enhancing Diversity and Reducing Bias in Recruitment through AI	Fairness & Ethical AI in HR	Explainable AI, bias detection, fairness-aware ML models	Literature-based study	Focuses on ethics and transparency but lacks technical implementation for resume analysis.
2023	Secured Recruitment Management System with Data Retention Approach	Recruitment System Security	JWT authentication, RBAC, encryption, CSRF protection	Prototype recruitment platform	Focuses only on system security, not AI/NLP-driven analysis or ranking.
2023	Illuminating HR Insights: Unravelling Patterns through Data Analytics and Visualization	HR Analytics & Visualization	Data analytics, visualization using charts and dashboards	HR data analytics dataset	Provides visualization but lacks AI-driven scoring, NLP, or predictive modelling.
2021	Job Candidate Rank Approach Using Machine	Candidate Ranking	Regression and classification models; vector	Collected candidate resumes	ML-based ranking without NLP

	Learning Techniques		similarity for scoring	and job posts	context understanding ; lacks semantic embeddings.
2024	CV Recommendation Model Using Latent Dirichlet Allocation	Resume Recommendation & Topic Modelling	Latent Dirichlet Allocation (LDA), probabilistic topic modelling	Resume and job description text corpus	Captures topic-level similarity but fails to model fine-grained semantic relationships and contextual nuances compared to transformer-based methods.
2025	How AI-Generated Resumes Impact the Staffing Industry	Recruitment Analysis & AI Impact	Analytical study on AI-generated resume optimization	Industry reports and AI-generated resume samples	Highlights risk of over-optimization and authenticity issues but does not provide a concrete automated evaluation or ranking system.

Chapter 3

Limitation of Existing System

3. Limitation of Existing System

The recruitment and resume evaluation process in many organizations still relies on traditional methods or partially automated tools that are unable to provide efficient and intelligent candidate assessment. Although several Applicant Tracking Systems (ATS) and recruitment platforms have been developed to assist HR departments, many of these systems suffer from technical and operational limitations. These limitations reduce the effectiveness of the hiring process and may lead to inaccurate candidate selection. The following points describe the major limitations observed in existing recruitment and resume screening systems.

1. **Manual Screening and Subjectivity:** Traditional recruitment systems rely heavily on manual resume screening, which is time-consuming and requires significant effort when handling large volumes of applications. This process introduces subjectivity, as different recruiters may interpret resumes differently based on personal judgment. Such inconsistency can lead to biased decisions, rejection of qualified candidates and errors like overlooking key skills, affecting fairness and accuracy.

2. **Keyword Dependency:** Many Applicant Tracking Systems depend on keyword matching to filter resumes. While this automates basic screening, it fails to provide reliable evaluation. Candidates often use synonyms (e.g., "ML" vs "Machine Learning"), which these systems may not recognize, leading to incorrect rejection of qualified applicants and reduced accuracy.

3. **Lack of Semantic Understanding:** Existing systems cannot understand the context or semantic meaning of resume content, focusing only on word matching. Without advanced NLP, they fail to capture the true relevance of a candidate's experience, especially when exact keywords are missing, resulting in poor matching outcomes.

4. **Limited Feedback to Applicants**: Most platforms provide minimal feedback such as "shortlisted" or "rejected," without explaining the reasons. This lack of transparency prevents candidates from understanding weaknesses or improving their resumes, as they receive no insights on skills, scores or gaps.

5. **Absence of Skill Gap Analysis and Recommendations**: Traditional systems only screen candidates and do not analyse skill gaps between resumes and job requirements. They also fail to provide recommendations like courses or certifications, limiting opportunities for improvement and self-assessment.

6. **No Integration of Mock Interviews or Success Prediction:** Most platforms lack features for interview preparation, such as mock interviews, and do not provide predictions about selection chances. This reduces their usefulness and offers limited support to candidates during the hiring process.

7. **Inadequate Candidate Ranking Mechanisms:** Candidate selection is often based on simple filters like education, experience or keywords. Advanced techniques such as TF-IDF, semantic analysis or ML-based ranking are rarely used, making it difficult to accurately identify the most suitable candidates.

8. **Security and Privacy Concerns**: Recruitment systems handle sensitive personal data, but many lack strong security measures like authentication, access control and encryption. This increases the risk of data breaches, unauthorized access and misuse of information.

9. **Lack of a Unified Platform for HR and Students**: Existing systems are often fragmented, using separate tools for resume submission, screening and analytics. This creates inefficiencies, as recruiters and candidates must switch between platforms, resulting in a disjointed and less effective recruitment process.

Chapter 4

Problem Statement and Objectives

4. Problem Statement and Objectives

4.1 Problem Statement

In the contemporary recruitment landscape, organizations face an overwhelming volume of applications for each role, making manual resume screening impractical due to its time-consuming nature, lack of scalability and susceptibility to bias and inconsistency. Recruiters must process large amounts of unstructured data under strict timelines, often leading to inefficient shortlisting and suboptimal hiring decisions.

To address this, Applicant Tracking Systems (ATS) were introduced, but most rely on keyword-based matching, failing to capture semantic meaning and contextual relevance. As a result, qualified candidates may be overlooked due to terminology differences, while keyword-heavy resumes may receive unfairly high rankings. Additionally, these systems provide limited transparency and feedback, leaving candidates unaware of rejection reasons and recruiters with low trust in automated decisions.

Another key challenge is the absence of skill gap analysis, as traditional systems do not effectively identify missing competencies or measure alignment with job requirements. Moreover, recruitment processes such as parsing, matching, ranking and explainability are often handled separately, leading to fragmented workflows and reduced efficiency.

Therefore, there is a need for a unified, intelligent system that integrates semantic understanding, interpretable scoring, skill gap analysis and transparent ranking within a single framework. The proposed solution addresses this by designing an NLP-based Resume Analysis and Ranking System that goes beyond keyword matching to provide accurate, context-aware, explainable and efficient recruitment support.

4.2 Objectives

The primary objective of this project is to design and develop an AI-powered recruitment system that automates resume analysis and candidate ranking using Natural Language Processing (NLP) and Machine Learning (ML). The system aims to bridge the gap between candidates and employers by providing intelligent, fair and data-driven hiring decisions.

The specific objectives of this project are as follows:

1. To develop an automated resume analysis system using NLP and machine learning techniques

The primary objective is to design and implement a system that eliminates the inefficiencies of manual resume screening by leveraging Natural Language Processing (NLP) and Machine Learning (ML). The system aims to process unstructured resume data and convert it into meaningful, structured insights for evaluation.

2. To overcome limitations of traditional keyword-based Applicant Tracking Systems (ATS)

The system aims to address the shortcomings of conventional ATS platforms that rely heavily on keyword matching. By incorporating semantic similarity techniques, the objective is to ensure deeper contextual understanding of candidate profiles rather than superficial keyword presence.

3. To design a reproducible and interpretable ATS scoring mechanism

A key objective is to build a deterministic and transparent scoring framework that combines multiple evaluation parameters such as keyword relevance, experience similarity, formatting quality, skill match and educational alignment. This ensures consistency, fairness and reproducibility in candidate evaluation

4. To enable semantic job-resume matching using advanced embedding techniques

The system seeks to improve candidate-job alignment by utilizing sentence-transformer models and cosine similarity measures. This allows the system to capture contextual meaning and relevance between resumes and job descriptions beyond lexical similarity.

5. To implement a hybrid skill gap detection module for candidate improvement

Another objective is to identify missing skills in a candidate's profile using a combination of ontology-based expansion and zero-shot natural language inference. This helps in providing accurate and interpretable insights into skill deficiencies relative to job requirements.

6. To provide actionable and structured feedback for student self-assessment

The system is designed to generate meaningful feedback rather than just numerical scores. The objective is to guide students with clear suggestions such as improving formatting, adding measurable achievements, or acquiring specific skills to enhance employability.

7. To develop a recruiter-side candidate ranking system with explainability

The objective includes building an intelligent ranking mechanism that combines structured candidate features with semantic similarity scores using a neural ranking model. Additionally, explainability is ensured through SHAP-based interpretations to support recruiter decision-making.

8. To ensure transparency and reduce bias in recruitment processes

By incorporating explainable AI techniques and deterministic scoring, the system aims to make hiring decisions more transparent and less prone to human bias, thereby improving fairness in candidate evaluation.

9. To maintain data privacy through local system deployment

The system is designed to operate locally using technologies like FastAPI, FAISS and SQLite, ensuring that sensitive candidate data is not exposed to external APIs. This objective focuses on secure, privacy-preserving recruitment workflows.

10. To support scalable and efficient handling of large volumes of applications

The system aims to handle bulk resume uploads and process multiple applications efficiently using asynchronous processing and optimized similarity search mechanisms, thereby improving recruitment scalability.

11. To provide dual functionality for both students and recruiters through dedicated portals

The objective is to create a unified platform that caters to both students and HR professionals. While students receive feedback and improvement insights, recruiters gain access to ranked candidate lists and structured evaluation metrics.

12. To align the system with real-world hiring challenges and global development goals

The system is developed to address practical recruitment issues such as time consumption, lack of transparency and inefficiency, while also contributing to broader goals like Quality Education (SDG 4) and Decent Work and Economic Goal (SDG 8).

4.3 Scope

The scope of the NLP-Based Resume Analysis and Ranking System extends across the complete recruitment lifecycle for both candidates and recruiters. The system aims to create an intelligent, automated and transparent platform that enhances employability and streamlines hiring.

For Students and Job Seekers:

Resume Analysis & ATS Scoring: Students can upload their resumes (PDF/DOCX) to receive a reproducible 0–100% ATS score. The score is calculated using five weighted components: keyword relevance (30%), experience matching (25%), formatting quality (20%), skill alignment (15%) and education fit (10%), enabling objective evaluation of resume quality. The score is calculated using five weighted components: keyword relevance (30%), experience matching (25%), formatting quality (20%), skill alignment (15%) and education fit (10%), enabling objective evaluation of resume quality.

Skill Gap Detection: The system identifies matched and missing skills by comparing resumes with job descriptions using ontology-based expansion and zero-shot NLI classification. It also provides confidence scores, helping students understand which skills need improvement.

Actionable Feedback: Students receive structured and practical suggestions such as adding measurable outcomes, improving formatting, inserting a dedicated skills section and quantifying achievements, based on rule-based diagnostics.

Course Recommendations: Based on detected skill gaps, the system suggests relevant upskilling resources through an external LLM-based recommendation service, helping students align their profiles with industry requirements.

Mock Interview Preparation: Students can access a curated interview question bank retrieved using semantic search, with optional LLM-generated role-specific questions to enhance preparation.

Application Tracking: A dashboard allows students to track application progress, including status transitions (submitted → shortlisted → interviewed → offered), activity timelines and average ATS scores across multiple resumes.

Privacy-First Design: All core processing runs locally using FastAPI, SQLite and FAISS, ensuring data security, privacy and reproducible results without reliance on external APIs for scoring.

For HR Professionals

Job Posting Management: Recruiters can create and manage job vacancies through a secure RESTful interface. The system ensures data validation using Pydantic models and implements JWT-based role access control for secure operations. It also performs automated skill and degree extraction from job descriptions using JobBERT-based Named Entity Recognition (NER), reducing manual effort in defining requirements.

Bulk Resume Processing: The platform supports bulk resume uploads through a drag-and-drop ZIP feature, allowing recruiters to process multiple PDF/DOCX files simultaneously. It includes duplicate detection using encrypted hashing and performs asynchronous processing with an average time of approximately 4.5 seconds per resume using local PyPDF2-based text extraction.

Candidate Ranking & Explainability: Candidates are ranked using a neural ranking model (RankNet) that combines structured features such as ATS score, experience, skill match, education, location and salary fit with semantic similarity measures. The system also provides SHAP-based explanations, enabling recruiters to understand feature-level contributions for top-ranked candidates and ensuring transparency in decision-making.

Application Pipeline Management: The recruitment workflow is managed using a finite-state machine (FSM) that controls application status transitions, including SUBMITTED → SHORTLISTED → INTERVIEW_SCHEDULED → INTERVIEWED → OFFERED, REJECTED, or WITHDRAWN. This ensures consistency in process flow, along with audit logging and Redis-based event propagation for real-time updates.

Interview Scheduling: Recruiters can schedule interviews using dedicated endpoints that handle date, time and mode of interview. This functionality is designed independently from application state transitions to maintain modularity and flexibility in managing interview processes.

HR Dashboard & Analytics: The system provides a visual dashboard that presents insights such as application trends, ATS score distributions and overall recruitment metrics. It also enforces row-level data isolation, ensuring that recruiters can only access and manage their own job postings and candidate data.

Deterministic & Auditable Evaluation: All scoring and ranking processes are executed locally using pre-trained models in evaluation mode, ensuring consistent and reproducible results. This eliminates variability and reduces bias while maintaining full auditability of the evaluation process.

Data Security & Compliance: The platform ensures secure data handling through ACID-compliant SQLite operations and enforces composite unique constraints to prevent duplicate applications. Additionally, it maintains comprehensive audit trails for all state changes, supporting transparency and compliance requirements.

Chapter 5

Proposed System

5. Proposed System

This section presents the methodology of the proposed NLP-Based Resume Analysis and Ranking System, which supports two primary user roles—Student and HR—through dedicated web portals. The system follows a modular architecture, enabling efficient execution of resume screening, semantic matching, ranking and feedback generation while ensuring scalability and maintainability. It processes inputs such as resumes and job descriptions using NLP techniques to extract key features like skills, experience and qualifications.

The workflow includes data input, processing, analysis and output generation. For students, the system computes ATS scores, identifies skill gaps and provides actionable feedback. For HR, it performs candidate evaluation, semantic matching and ranking based on both structured data and contextual similarity. By moving beyond keyword matching, the system captures deeper relationships between resumes and job descriptions, ensuring accurate and explainable ranking outcomes.

Figure 1 illustrates the system context, showing interactions between students, recruiters and the core system. Students upload resumes and receive ATS scores and skill-gap insights, while HR provides job descriptions for candidate evaluation and ranking.

The central system handles all processing tasks such as parsing, feature extraction, semantic analysis and ranking, delivering outputs like scores, rankings and insights. This high-level design ensures a structured, efficient and transparent recruitment workflow.

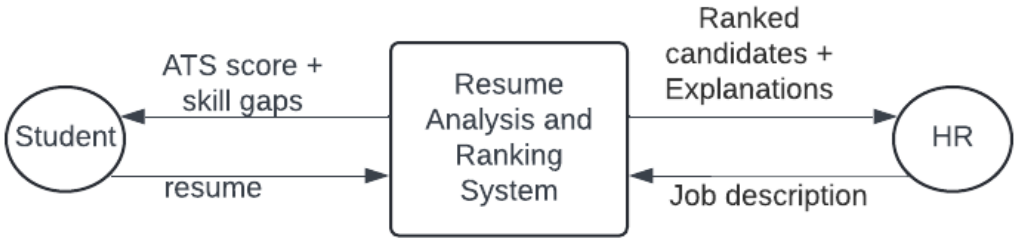


Figure 1: System Context Diagram

5.1 System Overview

The proposed system automates candidate evaluation using Natural Language Processing (NLP) and Machine Learning (ML) to analyse resumes against job descriptions. It functions as an end-to-end pipeline that processes unstructured data, extracts features and generates outputs for both students and HR users. Students receive ATS scores with feedback such as skill gaps and improvements, while HR users get ranked candidate lists with explainable insights. The system operates on a locally deployed AI engine, ensuring data privacy, real-time processing and independence from external APIs, making it suitable for secure and scalable recruitment.

The pipeline begins with data input from both users. Student resumes (PDF) and HR job descriptions are pre-processed through text extraction and cleaning. The processed data is then passed to the central Analysis and Ranking Engine, which performs feature extraction, semantic similarity computation and candidate evaluation. This engine interacts with the Feature and Embedding Store, where structured data and vector embeddings are maintained using SQLite and FAISS for efficient retrieval.

Figure 2 shows the system architecture and workflow. Inputs from students and HR are pre-processed and sent to the Analysis and Ranking Engine, which generates ATS scores, identifies skill gaps and ranks candidates. The Feature and Embedding Store supports this process by storing embeddings and processed data. Outputs include ATS scores and feedback for students,

and ranked candidate lists with explanations for HR, illustrating a centralized and efficient data flow.

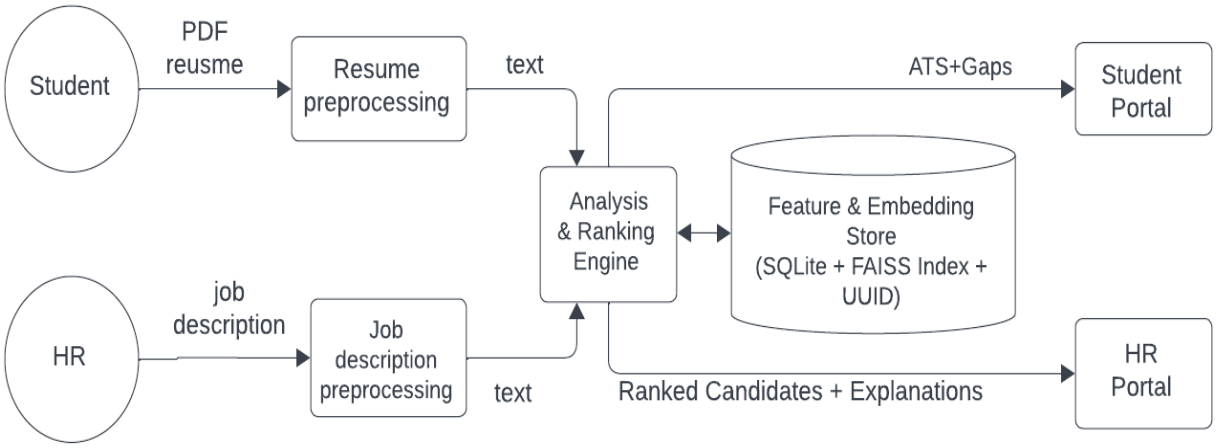


Figure 2: Functional Architecture Diagram

5.2 Student Modules

5.2.1 Job Browsing & Application Module:

The Job Browsing and Application Module enables students to search and apply for jobs through a RESTful service that supports keyword filtering with optional semantic enhancement. It uses a relational database with indexed fields for fast retrieval and supports pagination and filters such as role and skills.

An optional semantic re-ranking step improves results by embedding resumes and job descriptions into a 384-dimensional vector space and applying cosine similarity for better contextual matching.

The application process ensures consistency and data integrity through an authenticated and idempotent write path, preventing duplicate submissions. An ATS score is computed during submission, and data is stored using ACID-compliant transactions. A Redis stream event is generated for analytics, and all interactions are logged. The system maintains an average response time of around 650ms under light load, ensuring a smooth user experience.

5.2.2 Resume Ingestion and Parsing Module

The Resume Ingestion and Parsing Module handles the extraction and structuring of resume data for analysis. It supports PDF and DOCX formats using tools like PyPDF2 and python-docx, converting unstructured resumes into machine-readable text.

The extracted text is pre-processed by removing extra spaces, converting to lowercase and normalizing Unicode characters (NFKD). Unnecessary special characters are removed while retaining essential punctuation, improving data quality for NLP tasks.

The module then performs rule-based section segmentation using regex to identify sections like Education, Experience, Skills and Projects. Basic skill extraction is done using keyword matching, and both original files and processed data are stored in a SQLite database with timestamps for traceability.

5.2.3 Job-Match Analysis Module

The Job-Match Analysis Module computes the compatibility between a student's resume and a job description by generating a weighted ATS score. It evaluates five attributes: keyword relevance, resume formatting, experience relevance, skill matching and educational alignment, each contributing a fixed weight for balanced and interpretable scoring. Keyword relevance carries the highest weight (30%) for lexical similarity, followed by experience relevance (25%) for semantic alignment. Resume formatting contributes 20% to ensure structural clarity, skill matching adds 15% based on required skills and educational alignment accounts for 10% to verify eligibility. All component scores are normalized and combined using a fixed weighted aggregation method, ensuring consistent, deterministic and interpretable evaluation without relying on external APIs.

Let the normalized component scores be defined as: K:Keyword relevance score, F:Resume formatting score, E: Experience semantic similarity score, S: Skill match score, D: Education match score.

Keyword Relevance Score (TF-IDF Cosine Similarity)- The keyword score is computed using TF-IDF vectorization followed by cosine similarity between the resume text $R$and the job keyword document J:

$$K = \cos(\mathbf{v}_R, \mathbf{v}_J)$$

where $\mathbf{v}_R$and $\mathbf{v}_J$denote TF-IDF vectors constructed over unigrams and bigrams.

Formatting Score (Heuristic Aggregation)-

Resume formatting quality is assessed using rule-based heuristics capturing structural compliance:

$$F = \frac{H + B + L}{3}$$

where, H indicates the presence of required section headers, B denotes bullet-point density and L represents compliance with an optimal resume length range.

Experience Match Score (Semantic Similarity)-

The resume is split into overlapping text chunks $\{r_i\}$. Each chunk and the job description J are embedded into a shared vector space using a sentence-transformer model. The experience score is defined as the maximum cosine similarity:

$$E = max_i \cos(\mathbf{e}_{r_i}, \mathbf{e}_J)$$

This captures the strongest semantic alignment between any resume experience segment and the job requirements.

Skill Match Score-

Explicit skill overlap is computed as:

$$S = \frac{| S_r \cap S_j |}{| S_j |}$$

where S_r represents the set of skills extracted from the resume and S_j represents the required job skills.

Education Match Score-

Educational alignment is modelled as a binary score:

$$D = \begin{cases} 1, & \text{if a required degree keyword is present} \\ 0, & \text{otherwise} \end{cases}$$

Final ATS Score-

The overall ATS score is computed using fixed weights that sum to unity:

$$\text{ATS}_{score} = 0.30K + 0.20F + 0.25E + 0.15S + 0.10D$$

$$\text{ATS}_{\%} = \text{ATS}_{score} \times 100$$

Example:

Let the job description for a Junior AI Engineer be as follows:

Location: Mumbai (Hybrid)

Experience Level: Entry-Level (0–1 year)

Employment Type: Full-time

Required Skills: Python, Machine Learning, TensorFlow, Scikit-learn, OpenCV, SQL, Git, Problem Solving, Team Collaboration

Preferred Education: Bachelor of Engineering (B.E.) in Computer Science, Artificial Intelligence, or related field

Experience Expectation: Full-time, in-person internship or project experience in AI/ML

Let, the following resume be tested against the above mentioned job description:

Final-year B.E. in Computer Science & Engineering (AI & ML specialization) at APSIT, Mumbai University. Aspiring AI & ML enthusiast with practical experience from internships and academic projects, passionate about applying technology to solve real-world problems.

Skills**:** Python, Java, C, JavaScript, PHP, HTML, CSS, JS, PHP, MySQL, Scikit-learn, TensorFlow, Keras, CNN, Regression models, AWS (EC2, S3, Data Pipelines), ETL, SQL, OpenCV, Pandas, NumPy, Matplotlib, Git, Jupyter Notebook

Education: B.E. in Computer Science & Engineering (AI & ML), APSIT, Mumbai University (2021–Present, entering 7th semester).

Calculating the ATS Score for the above resume, against the given job description:

Keyword Relevance (K): TF-IDF cosine (resume, job_keywords) = 0.80

Formatting Score (F): (Header present + bullet density (0.85) + resume length (400–800)) / 3 = 0.90

Experience Relevance (E): max_i cos (MiniLM(chunk_i), MiniLM(job)) = 0.75

Skill Match (S): | {Python, ML, TensorFlow, …} ∩ required| / |required| = 0.75

Education Match (D): Degree keyword “B.E.” found, D = 1.0

Final ATS Score:

$ATS_{score} = (0.30 \cdot 0.80 + 0.20 \cdot 0.90 + 0.25 \cdot 0.75 + 0.15 \cdot 0.75 + 0.10 \cdot 1.0) = 0.8$

$ATS_{\%} = 0.8 \times 100 = 80\%$

5.2.4 Explainability and Feedback Layer

The Explainability and Feedback Layer enhances transparency by converting component scores (K, F, E, S, D) into meaningful, actionable feedback for students. Rather than displaying only numerical outputs, the system interprets these scores to generate insights. A rule-based formatting checker identifies structural issues such as missing sections or improper formatting, while a zero-shot Natural Language Inference (NLI) classifier detects missing job-relevant skills and assigns confidence scores. These outputs are translated into structured recommendations, such as improving formatting, adding sections, or incorporating measurable outcomes, and are presented alongside the ATS score.

The explainability mechanism ensures clarity without exposing complex model internals by relying on rule-based heuristics and semantic diagnostics. Formatting feedback is derived from structural checks, while skill-related gaps are identified through contextual analysis. This approach maintains consistency, reproducibility and interpretability.

Finally, feedback is presented in a structured format, highlighting key areas for improvement and providing actionable recommendations. Identified skill gaps are accompanied by confidence indicators, enabling students to prioritize enhancements and improve resume quality prior to recruiter-side evaluation.

5.2.5 Skill Gap Module

The Skill Gap Detection Module evaluates a student's resume to identify present and missing job-relevant skills for a given role. It begins by expanding required job skills using a predefined skill ontology, capturing parent–child relationships to include related or equivalent skills and improve analysis beyond exact keyword matching.

The system then performs case-insensitive matching between the resume and the expanded skill list to identify explicitly mentioned skills. For skills not detected, a semantic evaluation stage determines whether they are implicitly represented.

This is achieved using a zero-shot Natural Language Inference (NLI) classifier, which estimates the likelihood that a skill is not expressed in the resume. By combining ontology-based expansion with semantic inference, the module follows a hybrid, interpretable and training-free

approach. The output includes matched skills, missing skills and confidence scores, providing clear and actionable insights.

Let:

- Rdenote the resume text
- $S_j$denote the set of job-required skills
- O denote the skill ontology containing parent–child relations

Skill Ontology Expansion-

The required skill set is expanded using ontology traversal:

$$S_j^+ = S_j \cup \{ p \mid (p \rightarrow c) \in O \wedge c \in S_j \}$$

where S_j^+represents the ontology-expanded skill set.

Skill Presence Detection-

A skill is considered present if it occurs explicitly in the resume text:

$$S_{\text{present}} = \{s \in S_j^+ \mid s \subset R\}$$

Skill Gap Identification-

The missing skill set is defined as:

$$S_{\text{missing}} = S_j^+ \setminus S_{\text{present}}$$

Missing Skill Confidence Estimation (Zero-Shot Classification)-

For each missing skill $s \in S_{\text{missing}}$, a zero-shot classifier calculates the likelihood that the resume misses the skill. Let $P(s \mid R)$ denote the entailment probability output by the classifier, where:

$$P(s \mid R) \in [0,1]$$

The confidence that the skill is absent is computed as:

$$C_{\text{missing}}(s) = 1 - P(s \mid R) \quad \text{where } C_{\text{missing}} : S_{\text{missing}} \rightarrow [0,1]$$

The module returns:

$$\{S_{\text{present}}, S_{\text{missing}}, C_{\text{missing}}\}$$

To provide actionable help apart from skill gap analysis, the system also provides a voluntary course recommendation layer that is performed strictly after the skill gap inference. For every missing skill that exceeds a predefined threshold limit, the module queries an external large language module-based recommendation service to recover upskilling resources.

The recommendation function is defined as:

$$\mathcal{C}: S_{\text{missing}} \rightarrow \{\text{course resources}\}$$

where $\mathcal{C}(s)$ gives a set of relevant learning resources corresponding to the missing skill s.

The recommendation component is architecturally decoupled from the core skill gap detection module, ensuring it operates independently without affecting skill detection, confidence scoring or ATS calculation. This separation preserves the integrity of the evaluation pipeline, keeping all analytical outputs deterministic, consistent and reproducible. By isolating this layer, the system ensures that recommendations do not alter the underlying assessment logic.

Since the recommendation module relies on an external API for generating upskilling suggestions, it is treated as a non-deterministic and replaceable component. This allows flexibility in updating or substituting the service without impacting core functionalities. As a result, the system maintains auditability and reproducibility in evaluation while still providing dynamic recommendations as an auxiliary feature.

5.2.6 Mock Interview Module

To enhance coverage and role specificity beyond static interview question sets, this module integrates an external large language model to dynamically generate role-specific interview questions based on job roles and skill profiles. The generated questions are filtered for relevance and quality, then embedded using the same sentence-transformer model as earlier stages, ensuring seamless integration with the semantic retrieval framework.

This augmentation layer functions only as a content enhancement component and does not affect core system logic, including retrieval, ranking or similarity computations. As it relies on an external API, it is treated as a non-deterministic and replaceable module, similar to the recommendation component. This design preserves reproducibility, auditability and consistency while enabling flexible and adaptive content generation.

5.2.7 Application Tracking & Progress Monitoring Module

The module aggregates student interactions and evaluation outputs to provide a consolidated view of resume usage and application progress. It computes key metrics such as the number of resumes, total applications, status transitions and average ATS performance, enabling students to track progress and assess profile effectiveness over time.

Application progress is represented through status-wise distribution (e.g., submitted, shortlisted, interviewed). All metrics are derived using deterministic aggregation on locally stored relational data, ensuring transparency, low-latency analytics and reproducibility.

Let a student have (N) resumes ($\{R_1, R_2, \ldots, R_n\}$) with corresponding ATS scores ($\{ATS_1, ATS_2, \ldots, ATS_n\}$) from the Job-Match Analysis Module. These scores are aggregated to compute overall performance metrics, offering a structured view of job search progress.

The average ATS score displayed on the dashboard is computed as the arithmetic mean of ATS scores obtained from each resume-job evaluation. Mathematically, it is represented as:

$$ATS_{\text{avg}} = \frac{1}{N}\sum_{i=1}^{N} AT\,S_i$$

5.3 HR modules

5.3.1 Job Posting Management Module:

The module provides a secure RESTful interface for creating and managing job vacancies. Incoming data is validated using FastAPI with Pydantic models to ensure correct formatting and constraints, while JWT-based role verification restricts access to authorized HR users, maintaining system security and integrity.

Validated job postings are stored using ACID-compliant database operations, with schema-level constraints such as positive values, valid salary ranges and restricted job status categories. Each posting is linked to the corresponding HR user, enabling ownership tracking and controlled updates.

An optional issuing flag allows immediate activation of job postings and triggers Redis stream events for asynchronous updates in downstream modules like semantic indexing. This ensures timely availability of job data while maintaining consistency and efficiency in recruitment workflows.

5.3.2 Application & Interview Management Module

The module manages the full lifecycle of job applications using role-based APIs, a deterministic finite-state machine (FSM) and transaction-safe persistence, ensuring accuracy and responsiveness. Students apply via POST /jobs/{job_id}/apply, where duplicate submissions are prevented through a composite unique constraint. After validation, the system computes the ATS score and stores the application with an initial status of SUBMITTED. Application progression is governed by a deterministic FSM with defined states, S = {SUBMITTED, SHORTLISTED, INTERVIEW_SCHEDULED, INTERVIEWED, OFFERED, REJECTED,

WITHDRAWN} and actions, A = {submit, shortlist, schedule_interview, record_interview, offer, reject, withdraw}. Terminal states such as OFFERED, REJECTED and WITHDRAWN do not allow further transitions. State updates are performed via PATCH /applications/{application_id}/status, accessible only to HR users. A transition function $\delta: S \times A \rightarrow S$ is enforced at the service layer to ensure only valid workflow transitions are allowed, rejecting invalid changes such as REJECTED → OFFERED. Interview scheduling is handled separately, where transitioning to INTERVIEW_SCHEDULED updates the state and detailed information such as date, time and mode is stored using a dedicated POST /applications/{id}/interview-details endpoint, maintaining separation of concerns.

The module enforces strict access control, allowing HR users to manage only their own job applications. All transitions are audit-logged with state changes, user ID and timestamps, and trigger events to a Redis Stream for real-time updates. A paginated GET /applications API supports the HR dashboard, enabling sorting and clear visualization of candidate progress through defined stages.

5.3.3 Bulk Resume Upload Module

The module allows HR users to process multiple resumes in a single operation, improving efficiency in high-volume recruitment. It supports ZIP uploads containing PDF or DOCX files, which are extracted and processed using PyPDF2 and python-docx. Each resume is passed through the same ATS scoring pipeline as individual evaluations, ensuring consistency.

To avoid redundant processing, an encrypted hash is generated for each resume and checked against existing records to detect duplicates. Processing is handled asynchronously as background tasks, preventing system slowdown and maintaining responsiveness during bulk uploads.

All extracted text and metadata are stored along with original files for reuse in ranking, matching and analytics. Initial testing showed an average processing time of about 4.5 seconds per resume, with further optimization planned for large-scale workloads.

5.3.4 Candidate Ranking & Explainability Module

After resume upload and ATS-based analysis, shortlisted candidates are further evaluated using a composite learning-based ranking model enhanced with semantic understanding. The primary objective of this module is to generate a stable, interpretable and well-ordered ranking of candidates while maintaining recruiter trust and clarity in decision-making. Unlike purely

rule-based systems, this module combines structured features with learned representations to capture both quantitative attributes and contextual relevance, resulting in a more balanced and reliable ranking outcome.

Each candidate is encoded as a fixed-length numeric feature vector:

$$\mathrm{x}_c = \begin{bmatrix} ATS_c, \\ YearsExp_{c,} \\ SkillMatch_{c,} \\ EduLevel_c, \\ LocationMatch_{c,} \\ SalaryFit_c \end{bmatrix}$$

where ATS_c denotes the deterministic resume–job compatibility score,$YearsExp_c$ represents years of professional experience, $SkillMatch_c \in [0,1]$ is the proportion of required skills matched, $EduLevel_c$ is an ordinal encoding of education level, $LocationMatch_c \in \{0,1\}$ indicates geographic compatibility and $SalaryFit_c$ models the alignment between the candidate's expected salary $s_c$and the job budget b. SalaryFit is clipped to [0,1] to ensure bounded values.

$$SalaryFit_c = 1 - \frac{|\, s_c - b \,|}{b + 1}$$

All features are standardized using precomputed mean and variance parameters to maintain numerical stability and consistent scaling across inputs.

Candidate relevance is computed using a lightweight feed-forward neural network based on the RankNet architecture:

$$f(\mathrm{x}) = W_2 \cdot ReLU(W_1\mathrm{x} + b_1) + b_2$$

This network outputs a scalar relevance score for each candidate, capturing non-linear relationships between features. The model parameters are pre-trained and loaded in evaluation mode during inference, ensuring deterministic and reproducible ranking behaviour without additional training overhead.

To enhance the ranking process with contextual understanding, semantic similarity between the job description and candidate resumes is computed using sentence-level embeddings. These embeddings are generated using a sentence-transformer model and FAISS is employed for efficient similarity computation across large candidate sets. This semantic score captures deeper alignment that may not be reflected in structured features alone.

The final ranking score is obtained through a weighted ensemble of the neural network output and the semantic similarity score:

$$Score_c = 0.7 \cdot f(\mathrm{x}_c) + 0.3 \cdot s_c^{sem}$$

where s_c^{sem} denotes the semantic similarity between the candidate and the job description. Candidates are then ranked in descending order of this combined score, ensuring that both structured attributes and contextual relevance contribute to the final decision. This hybrid approach results in a more accurate, interpretable and robust candidate ranking system.

Explainability via SHAP

SHAP (SHapley Additive exPlanations) is used within the HR Candidate Ranking module to provide transparent explanations of candidate relevance scores. For top-ranked candidates, SHAP DeepExplainer computes feature contributions—such as ATS score, experience, skill match and education—indicating their positive or negative impact on the final ranking. These values are converted into concise, human-readable insights for recruiters, ensuring clarity while preserving model privacy.

Although SHAP values are generated within the ranking pipeline, they are not directly visualized in the HR dashboard. Instead, the interface focuses on aggregate metrics like ATS distributions and application trends. SHAP operates as a backend interpretability layer, enhancing transparency without complicating the user interface.

To make sure of transparency, SHAP DeepExplainer is deployed to the trained RankNet for the top- listed candidates/users. For each input feature x_i , a SHAP value ϕ_i measures its contribution to the system output:

$$f(\mathrm{x}) = \phi_0 + \sum_{i=1}^{n} \phi_i$$

where ϕ_0 represents the base value and ϕ_i represents the contribution of each feature. These values are converted into feature-level signals (e.g., ATS influence, skill alignment, experience relevance) and summarized into natural-language explanations for easy interpretation.

The module outputs a ranked list of candidates with relevance scores, recommendation categories and concise explanations. All processing is local, ensuring low latency, reproducibility and data security while supporting clear and informed hiring decisions.

Chapter 6

Experimental Setup

6. Experimental Setup

This section outlines the experimental setup used to design, implement and evaluate the proposed system. It describes the data inputs, storage mechanisms and processing pipelines that support resume analysis and candidate ranking. Additionally, it presents the performance evaluation criteria used to assess system efficiency and reliability, along with the software and hardware environment in which the system operates.

6.1 Details of Database and Input to System

The system uses SQLite as its primary relational database, ensuring ACID-compliant transactions for consistency, reliability and data integrity. FAISS is integrated with a flat index for efficient similarity search over 384-dimensional sentence-transformer embeddings. UUID-based indexing ensures unique identification and traceability across entities, while tsvector indexing supports optimized text search for job descriptions and status-based queries.

Resumes in PDF and DOCX formats are processed using PyPDF2 and python-docx. Extracted text undergoes preprocessing including lowercasing, Unicode normalization (NFKD), removal of unnecessary special characters (while retaining meaningful punctuation) and whitespace

cleaning. This normalization improves consistency and downstream NLP performance. Both original files and processed text are stored with ISO 8601 timestamps to ensure auditability.

For real-time processing, Redis streams are used to emit structured events such as application_status_changed, enabling asynchronous workflows and analytics. Duplicate prevention is enforced via composite unique constraints on (student_profile_id, job_id), while bulk processing uses encrypted hash-based detection to eliminate duplicate resumes and avoid redundant computation.

6.2 Performance Evaluation Parameters

The system's performance is evaluated using key operational metrics from controlled testing. The Job Browsing and Application Module achieves an average latency of ~650ms under light load, ensuring responsiveness, while the Bulk Resume Upload Module processes resumes in ~4.5 seconds per file, indicating reliable throughput for moderate workloads.

Evaluation is based on a deterministic ATS scoring framework (0–100%), computed using fixed weights: Keyword Relevance (30%), Experience Relevance (25%), Formatting Quality (20%), Skill Match (15%) and Education Alignment (10%). Semantic operations are performed in a 384-dimensional embedding space using MiniLM-based sentence-transformers, ensuring consistent similarity computation.

Reproducibility is maintained by using pre-trained models in evaluation mode and executing all core components locally without external APIs. This ensures deterministic outputs, consistent performance and strong auditability, where identical inputs always produce identical results.

6.3 Software and Hardware Setup

Software Components:

• **Backend Framework (FastAPI):** Used to build the RESTful API with support for real-time inference and JWT-based role-based access control for HR users.
• **Frontend Interface (React with Chakra UI):** Provides a responsive and user-friendly interface for both students and recruiters.

Core NLP/ML Libraries:

- Sentence-Transformers (MiniLM): Generates 384-dimensional embeddings for semantic similarity using cosine similarity and FAISS.
- JobBERT: Performs NER to extract structured entities (skills, degrees, companies) from job descriptions.
- BART-based Natural Language Inference (NLI): Zero-shot classifier for skill gap detection using $C_{\text{missing}}(s) = 1 - P(s \mid R)$.
- TF-IDF with Cosine Similarity: Computes keyword-based relevance for ATS scoring.
- PyPDF2 and python-docx: Handles data validation and schema constraints.
- Pydantic: Handles data validation and schema constraints.

• Vector Search Engine (FAISS): Enables efficient similarity search and embedding indexing.

• Database System (SQLite): Provides ACID-compliant storage with timestamped records.

• Explainability Framework (SHAP – DeepExplainer): Computes feature-level contributions for ranking decisions.

• Event Streaming (Redis): Supports real-time event propagation and analytics through streams.

Hardware Specification:

The system is designed to run efficiently on standard hardware, making it suitable for academic and organizational environments while maintaining reliable performance for real-time and batch tasks. A typical configuration includes:

- **Processor:** Intel Core i5 (or equivalent), capable of handling concurrent API requests, lightweight ML inference and background processing tasks efficiently.
- **RAM:** 16 GB, ensuring smooth execution of multiple modules simultaneously, including embedding generation, resume parsing and batch processing without memory bottlenecks.

- **Storage:** At least 256 GB SSD, providing faster read/write speeds for database operations, model loading and file handling, which improves overall system responsiveness.

- **Operating System:** Windows 10/11, offering a stable environment for running the backend services, frontend interface and supporting libraries.

This configuration ensures that the system can operate with low latency, handle moderate workloads effectively and maintain consistent performance without requiring GPU acceleration or specialized hardware.

Chapter 7

Results and Discussion

7. Results and Discussion

7.1 Interpretation of Results

The system demonstrates that local, deterministic NLP pipelines can perform reproducible resume analysis without relying on external APIs. The weighted ATS scoring mechanism—keyword relevance (30%), experience (25%), formatting (20%), skills (15%) and education (10%)—ensures interpretable and auditable evaluation. Semantic matching using MiniLM embeddings captures contextual relevance beyond keyword matching, addressing key limitations of traditional ATS systems.

In terms of performance, the system achieves ~650ms browse latency and ~4.5 seconds per resume in bulk processing, indicating feasibility for moderate-scale deployment. However, the lack of benchmarking against commercial ATS platforms limits quantitative performance comparison.

7.2 Strengths of the Proposed System

The system ensures reproducibility and transparency through fixed scoring weights, deterministic aggregation and pre-trained models in evaluation mode. Its local deployment with FastAPI, SQLite and FAISS maintains data privacy by avoiding external APIs. The dual-role architecture supports both students and HR users, enhancing overall system utility.

A hybrid skill gap detection approach of combining ontology expansion with zero-shot NLI—enables interpretable, training-free identification of missing skills. SHAP-based explainability provides feature-level insights for top candidates, though currently limited to backend outputs without graphical visualization in the HR portal.

7.3 Limitations and Constraints

The system has several acknowledged limitations. Fixed scoring weights may not generalize across different industries or roles, and reliance on textual resume data limits support for multi-modal inputs such as portfolios or certifications. Additionally, dependence on predefined skill ontologies may reduce adaptability to rapidly evolving domains.

While local deployment ensures data privacy, it limits scalability for large-scale usage. SHAP-based explanations are computed but not visually presented in the HR interface, creating a gap between backend capability and frontend usability. Furthermore, evaluation is based on light-load testing, lacking large-scale validation and comparison with existing platforms.

7.4 Results and System Outputs

The proposed NLP-Based Resume Analysis and Ranking System generates structured, interpretable and role-specific outputs for both students and HR professionals, enabling automated evaluation and transparent decision-making.

Student-Side Results and Outputs:

For students, the system provides analytical outputs to improve employability and application quality:

- ATS Score (0–100%): Each resume is evaluated using weighted components—keyword relevance (30%), experience (25%), formatting (20%), skills (15%) and education (10%) offering an objective compatibility measure.
- Skill Gap Analysis: Identifies matched and missing skills with confidence scores using

zero-shot NLI, highlighting improvement areas.

• Actionable Feedback: Rule-based suggestions such as adding measurable outcomes or improving formatting structure.

• Course Recommendations: External LLM-based suggestions aligned with identified skill gaps.

• Mock Interview Questions: Generated via semantic retrieval, optionally enhanced for role-specific preparation.

• Application Tracking: Dashboard showing status transitions, timelines and average ATS scores.

• Privacy-Preserving Outputs: All results are generated locally (FastAPI, SQLite, FAISS), ensuring security and reproducibility.

HR-Side Results and Outputs

For HR professionals, the system provides decision-support outputs to improve efficiency and transparency:

• Job Posting Outputs: Structured listings with validated inputs and automatically extracted skills and degree requirements using JobBERT-based NER.

• Bulk Resume Processing: Simultaneous processing of multiple resumes with individual ATS scores and structured data extraction.

• Candidate Ranking: Ranked list based on a composite score combining ATS, experience, skill match, education, location, salary fit and semantic similarity.

• Explainability (SHAP-based): Feature-level insights showing contributions of factors like ATS, skills and experience for top candidates.

• Application Pipeline Tracking: Structured progression through states (SUBMITTED → SHORTLISTED → INTERVIEW_SCHEDULED → INTERVIEWED → OFFERED / REJECTED / WITHDRAWN) with audit logs.

• Interview Scheduling: Management of interview details via dedicated endpoints, separate from application state transitions.

• HR Dashboard & Analytics: Insights such as application trends, ATS distributions and recruitment progress, with secure row-level access control.

• Deterministic Outputs: Consistent scoring and ranking using locally deployed, pre-trained models.

• Secure & Auditable Data: ACID-compliant storage with constraints and audit trails ensuring reliability and traceability..

7.5 Practical Implications

For students, the system transforms ATS evaluation into a transparent self-assessment tool with actionable feedback, supporting SDG 4 (Quality Education) through interpretable evaluation mechanisms. For recruiters, it provides ranked candidate lists with structured explanations, supporting decision clarity while maintaining human agency in final hiring choices. At an institutional level, the system aligns with SDG 8 (Decent Work) through transparent and reproducible recruitment infrastructure.

7.6 Technical Trade-offs

The system architecture reflects deliberate trade-offs between key design priorities. Local deployment is chosen for data privacy, reproducibility and low latency, but limits scalability and access to elastic resources. Fixed scoring weights ensure interpretability and consistency, though they reduce adaptability across domains. SQLite provides simplicity and ACID compliance, but introduces concurrency constraints and restricts horizontal scaling. The use of zero-shot NLI avoids the need for training data and supports generalization, but may offer lower precision than fine-tuned models. The hybrid rule-based and neural design balances transparency with semantic understanding, though it increases system complexity and potential failure points.

7.7 Implementation

Figures 3 to 14 represent student-side output screenshots and Figures 15 to 20 represent HR-side output screenshots, illustrating the respective interfaces and analytical results generated by the system for each user role.

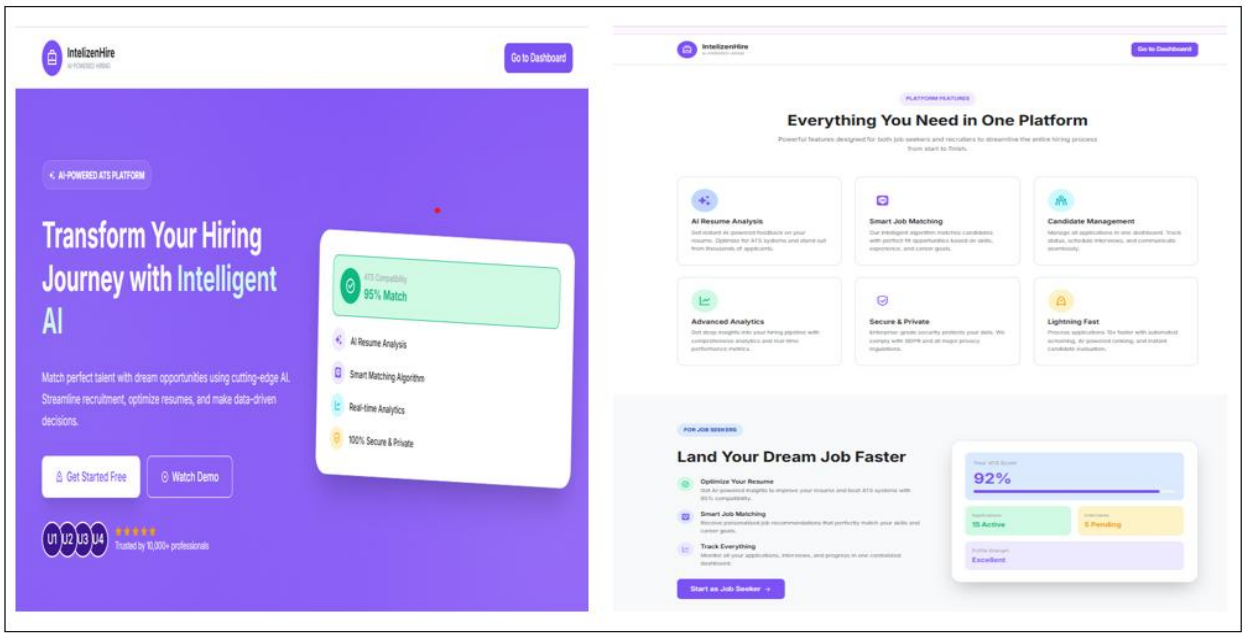


Figure 3: Landing Page

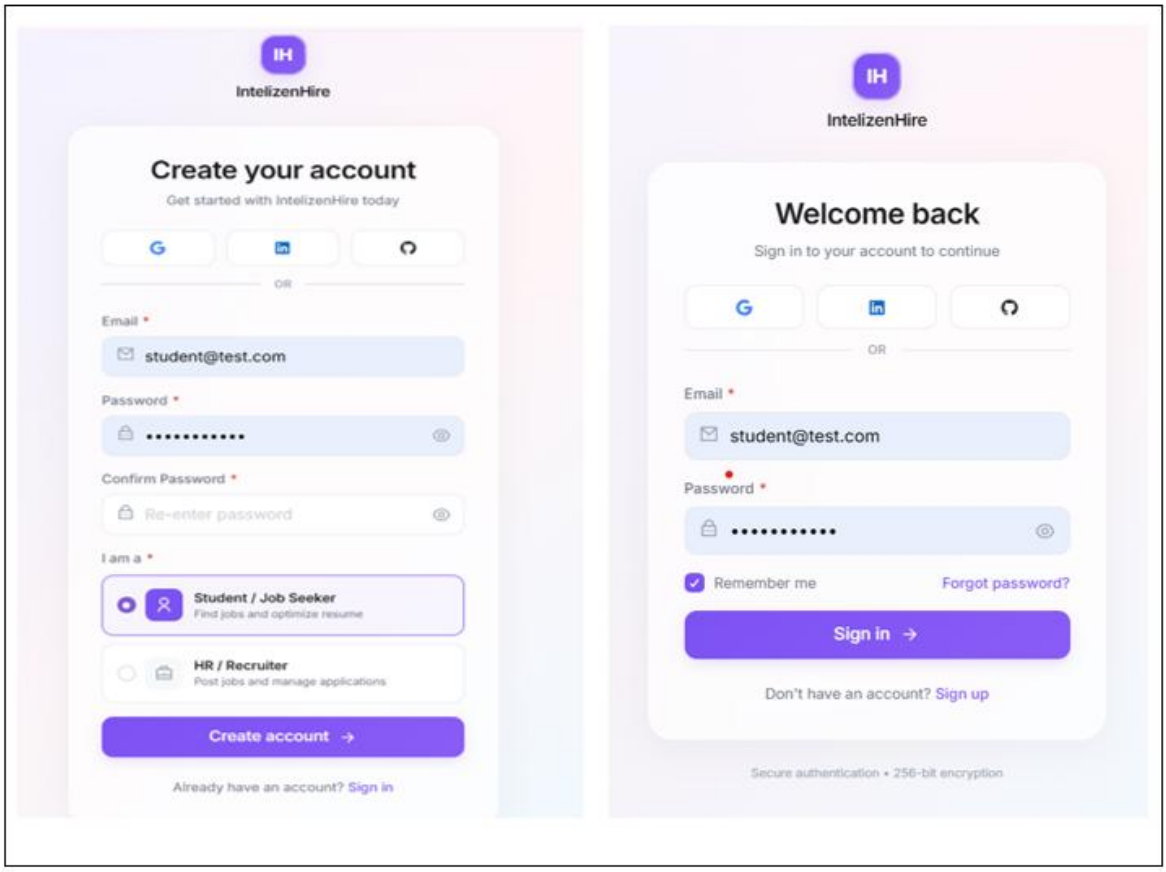


Figure 4: Register and Sign in Page

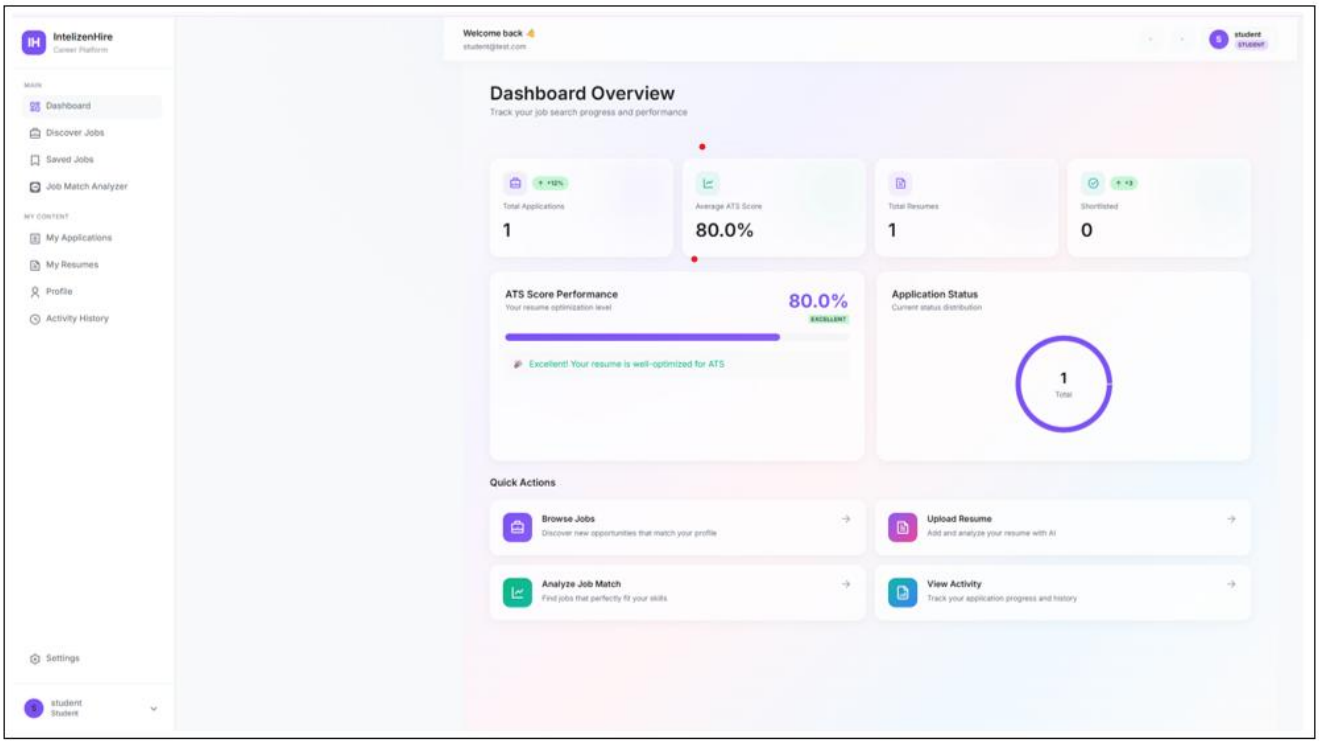


Figure 5: Student Dashboard

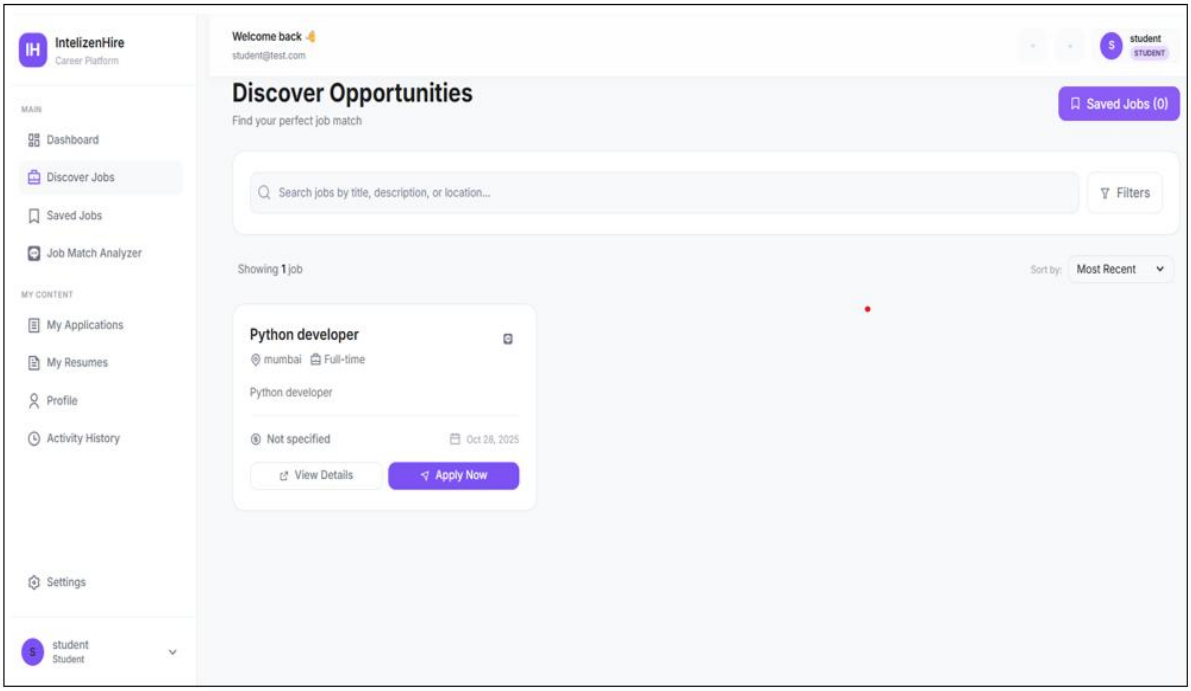


Figure 6: Student Job Browsing

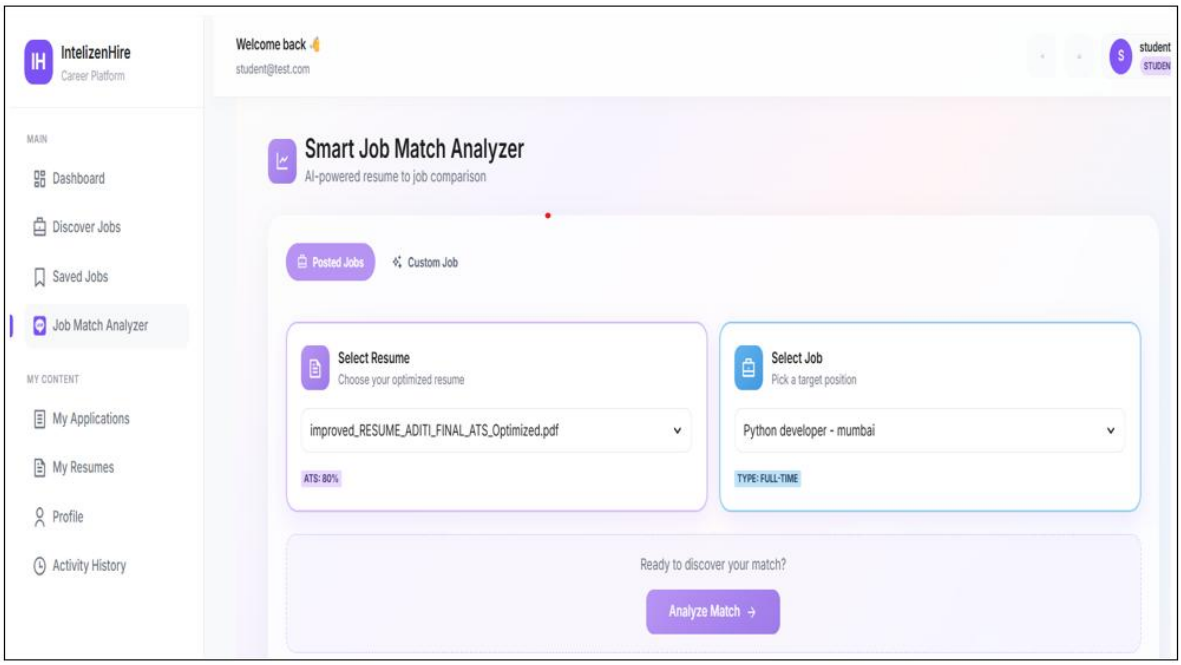


Figure 7: Student Resume & Job Matcher

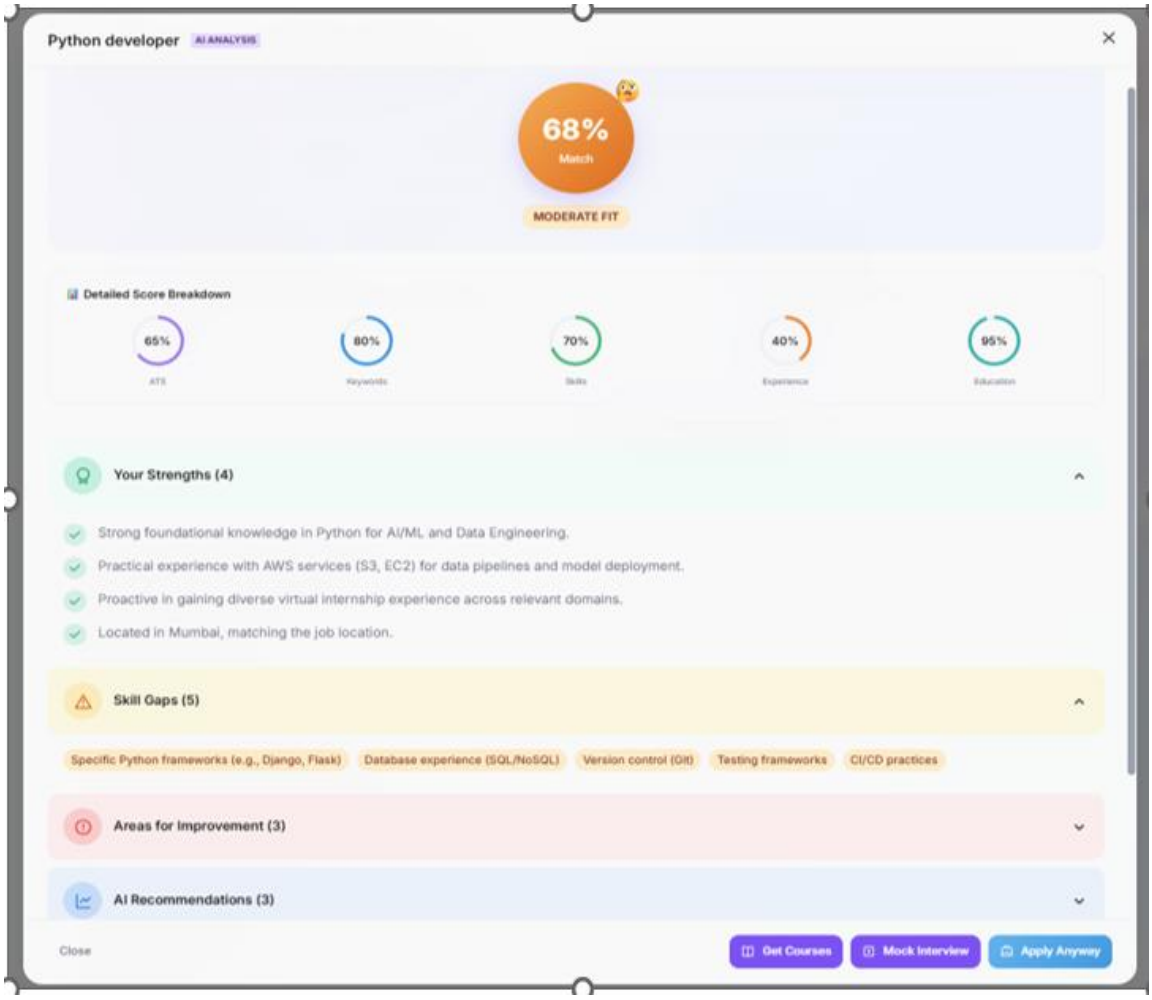


Figure 8: Student Resume Analysis

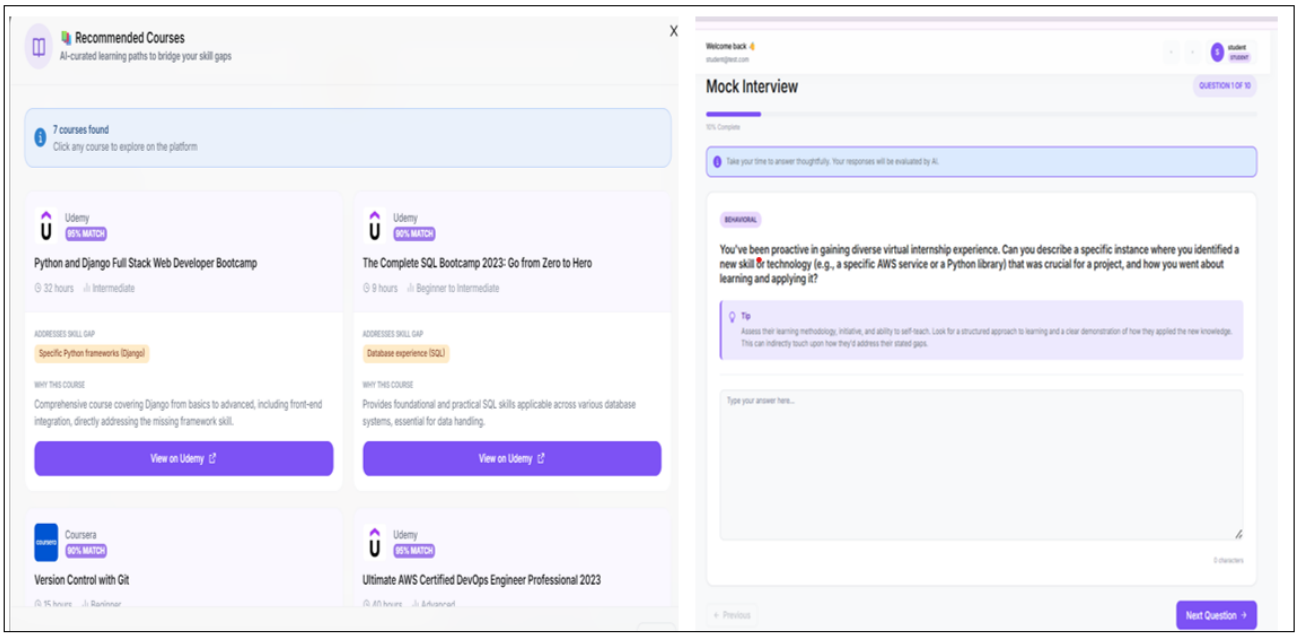


Figure 9: Student: Recommendation Courses & Mock Interviews

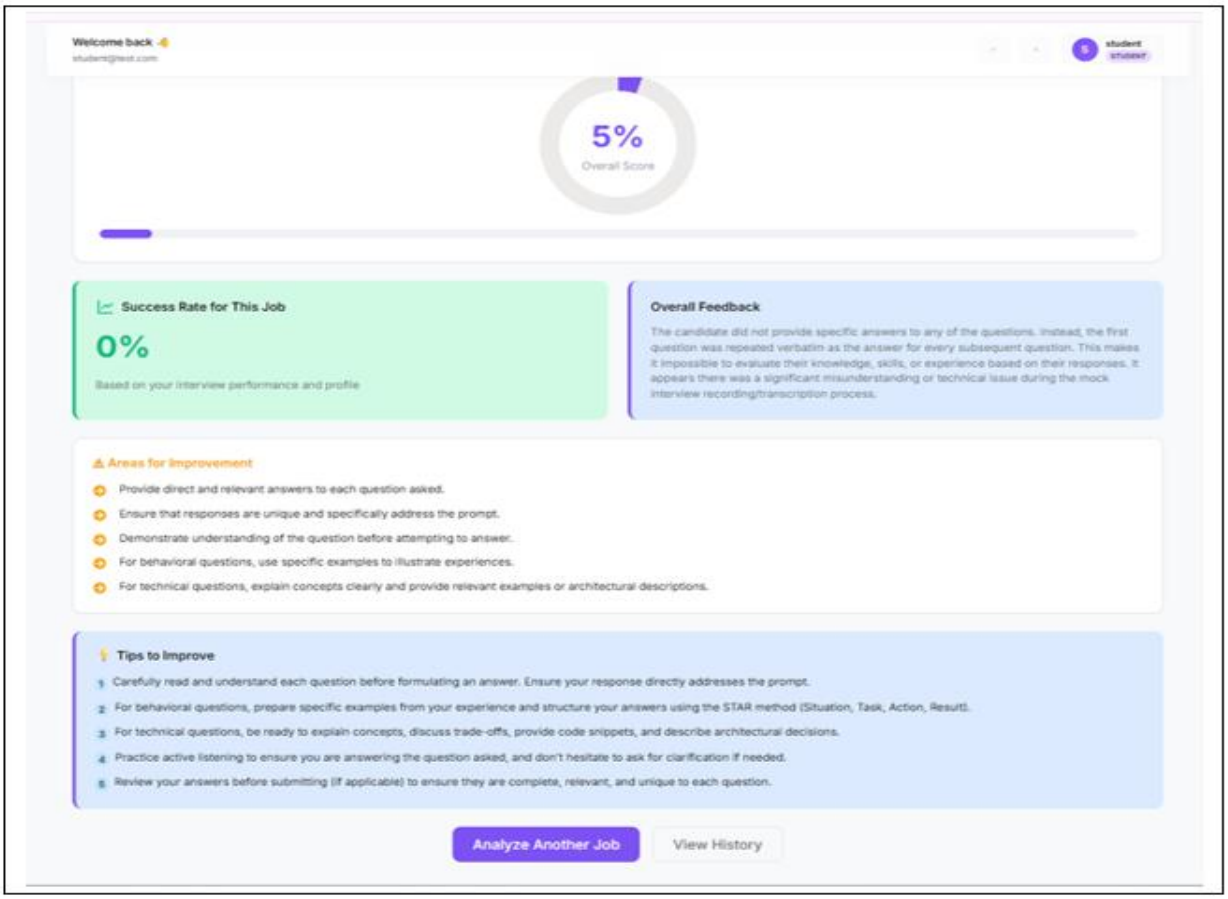


Figure 10: Job Matching Analysis

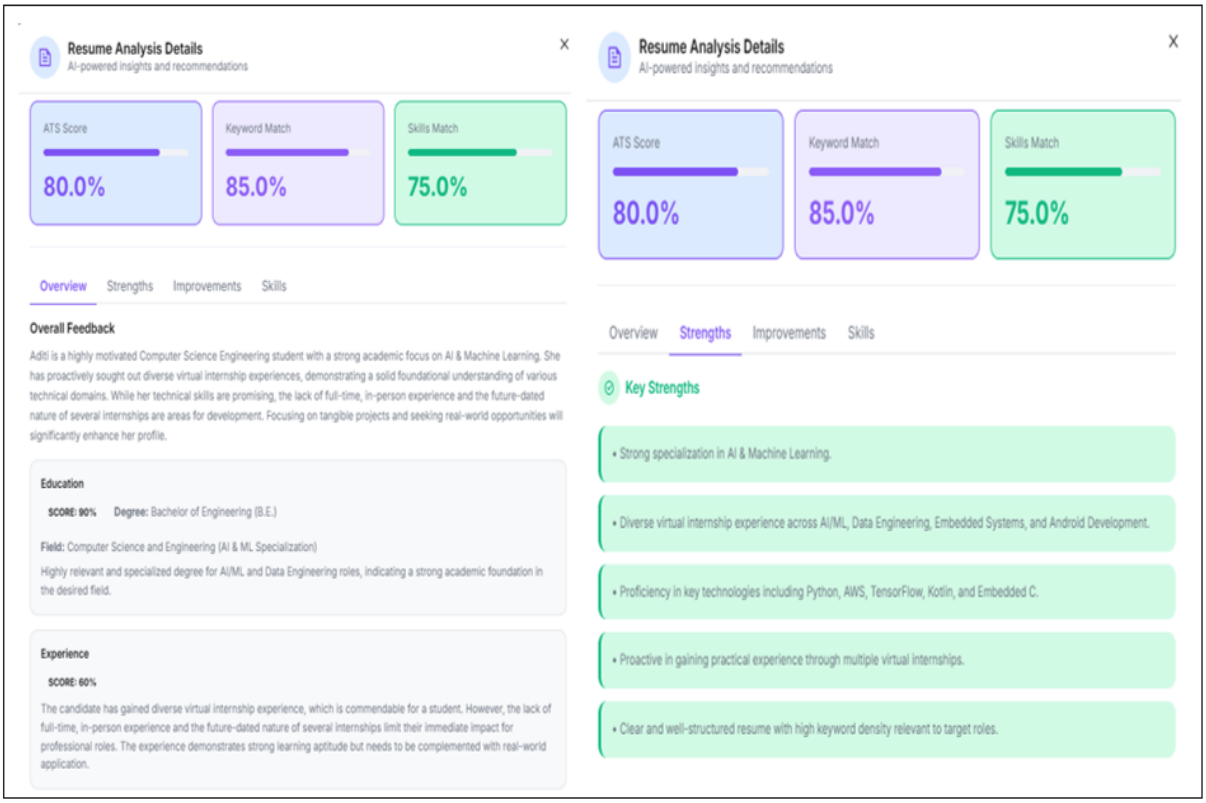


Figure 11: Student Resume Analysis (Overview & Strengths)

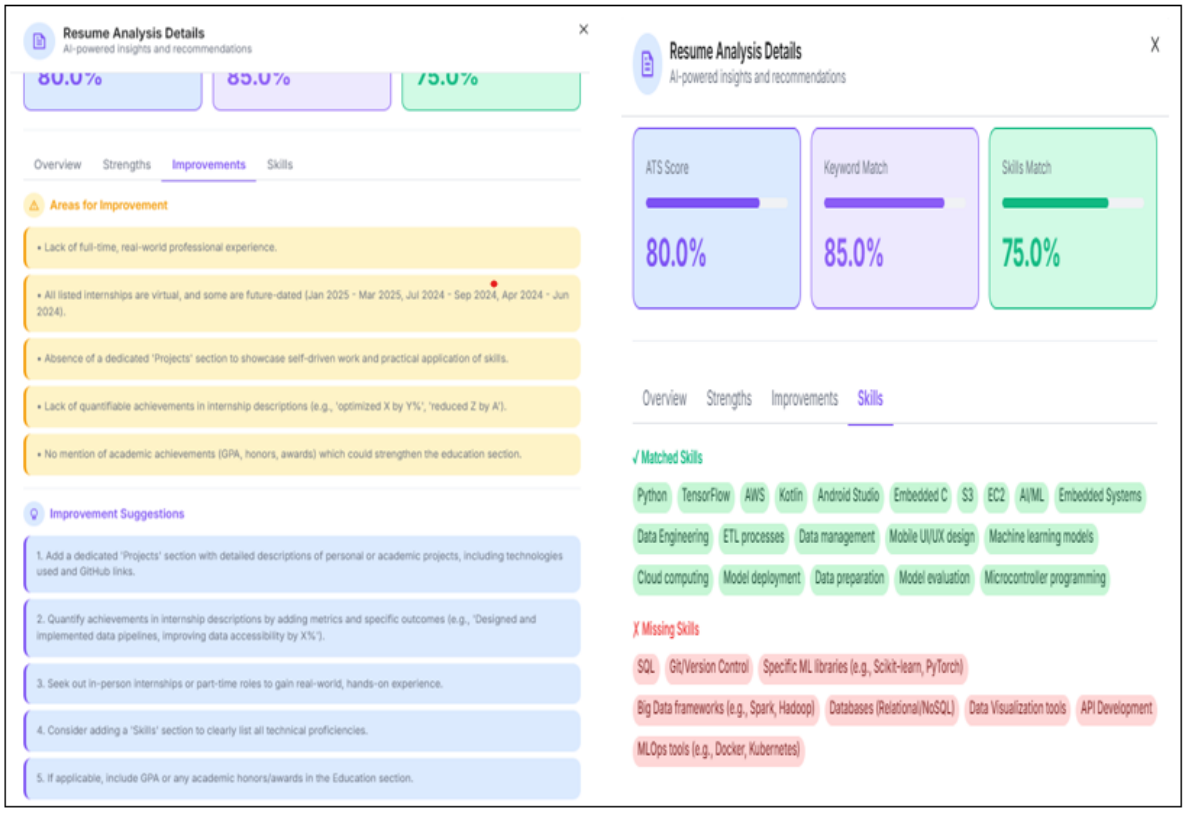


Figure 12: Student Resume Analysis (Improvements & Skills)

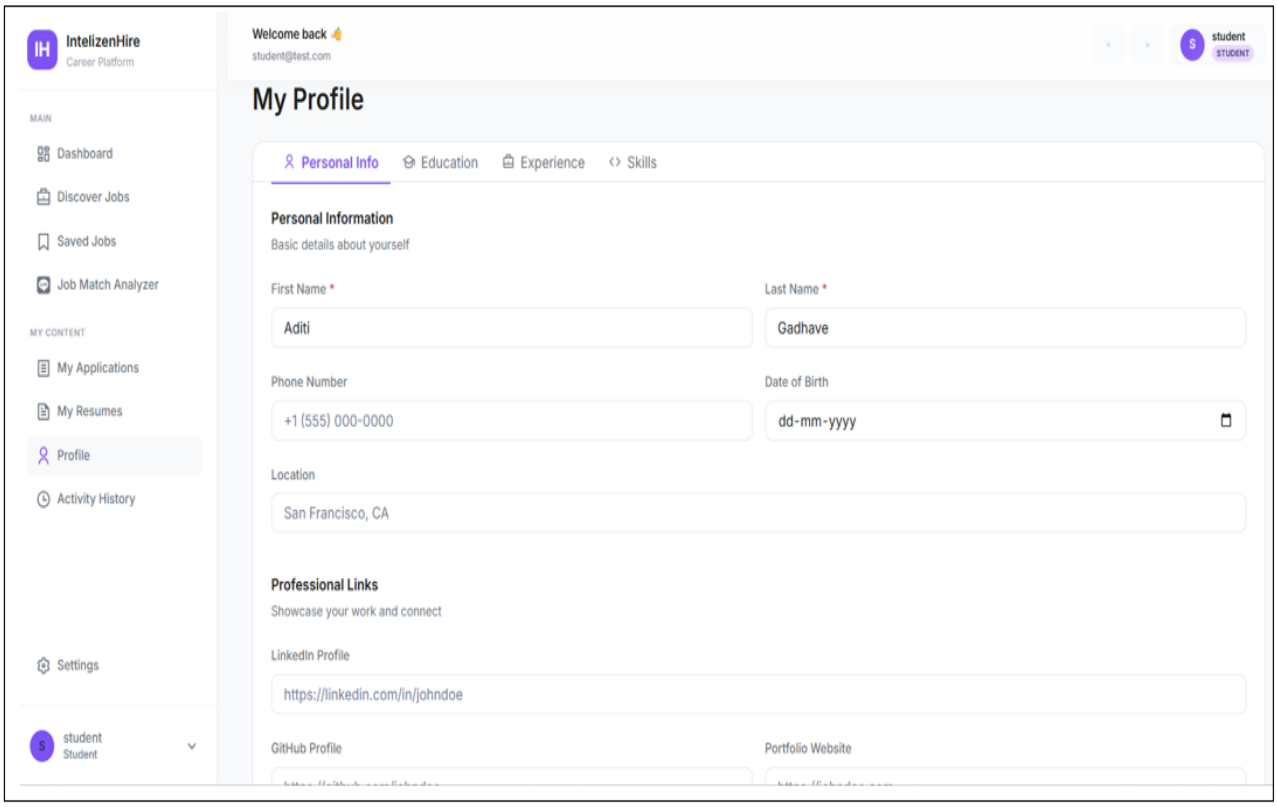


Figure 13: Student Profile Updation

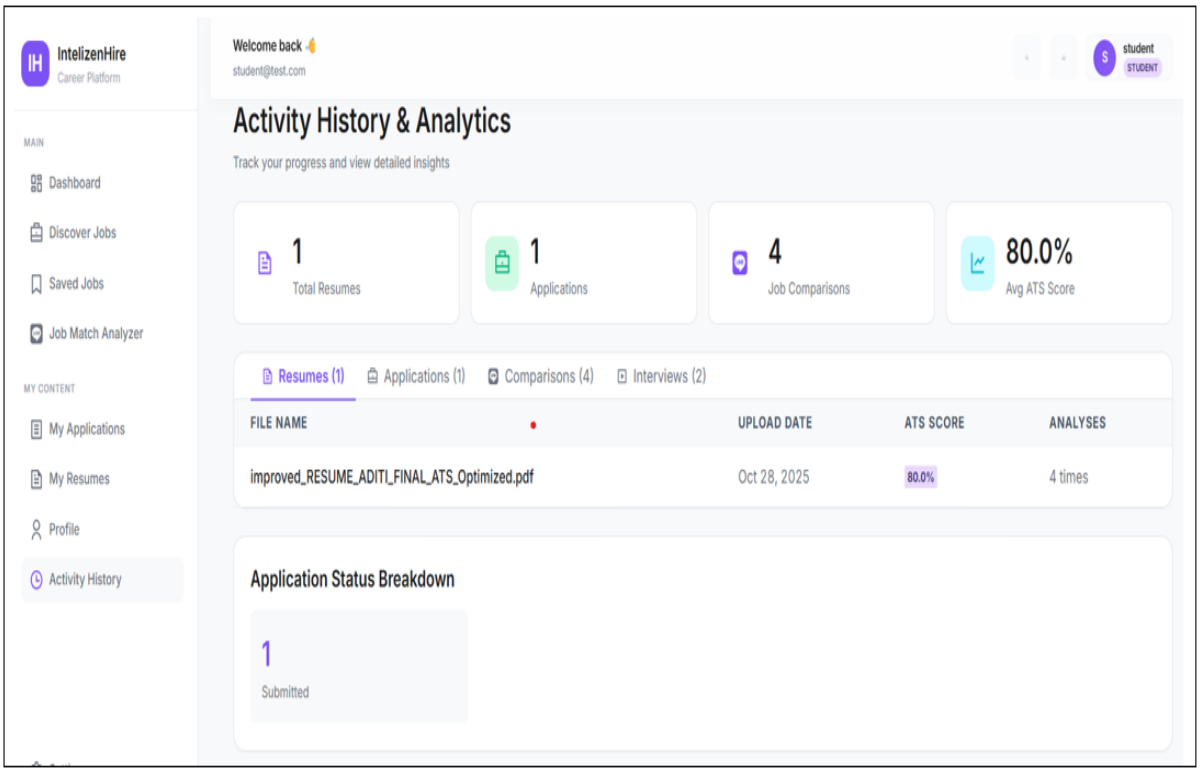


Figure 14: Student Activity History

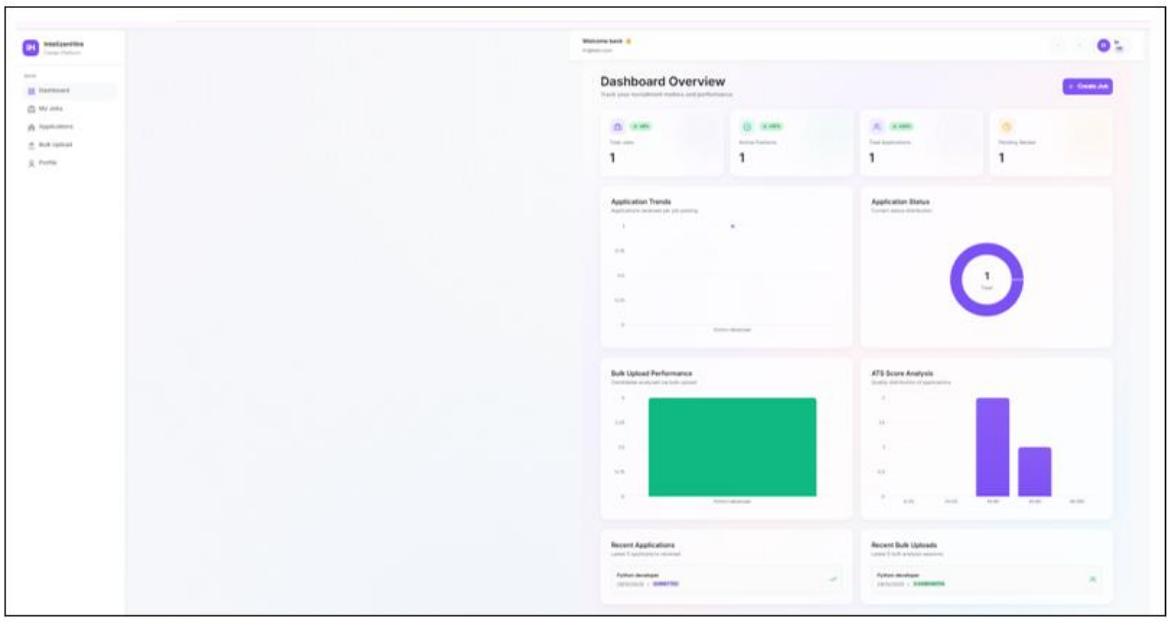


Figure 15: HR Dashboard

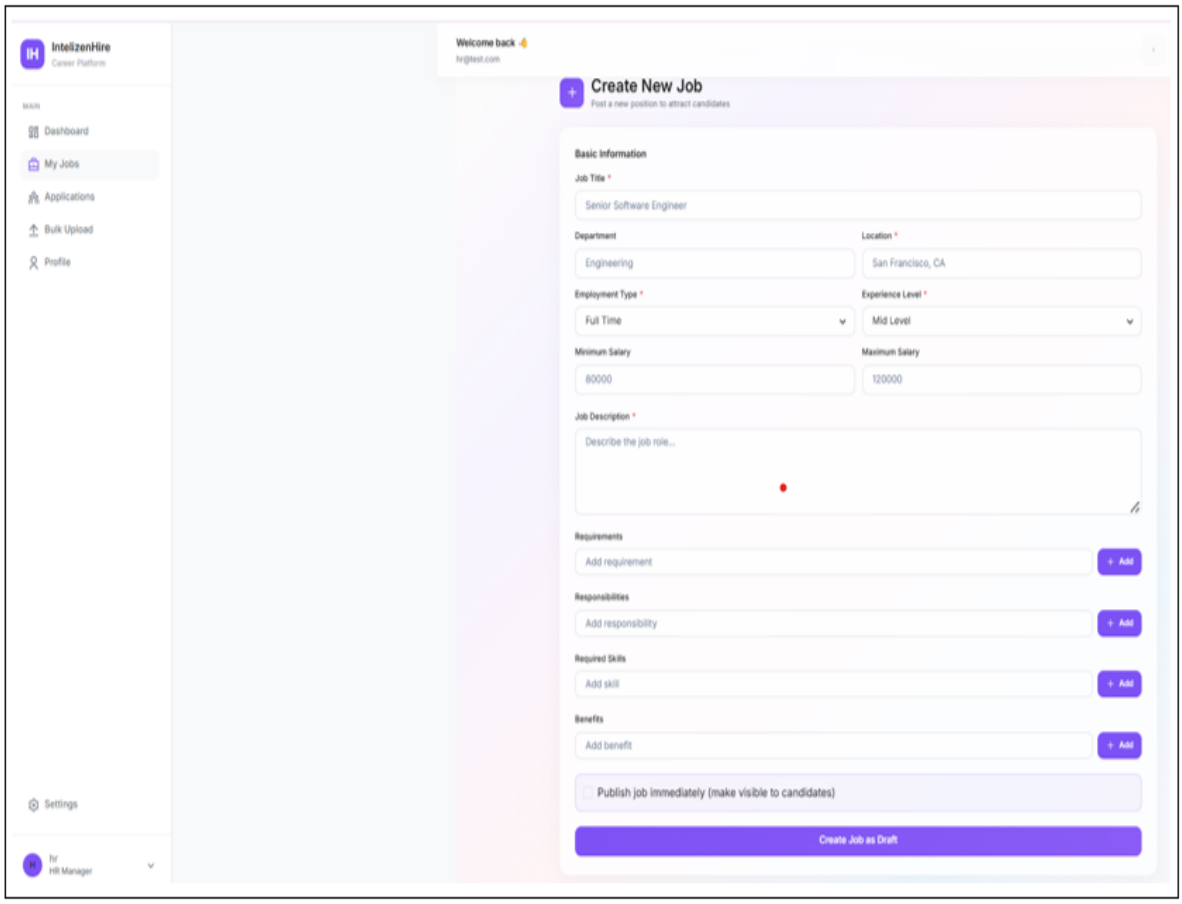


Figure 16: HR Job Posting

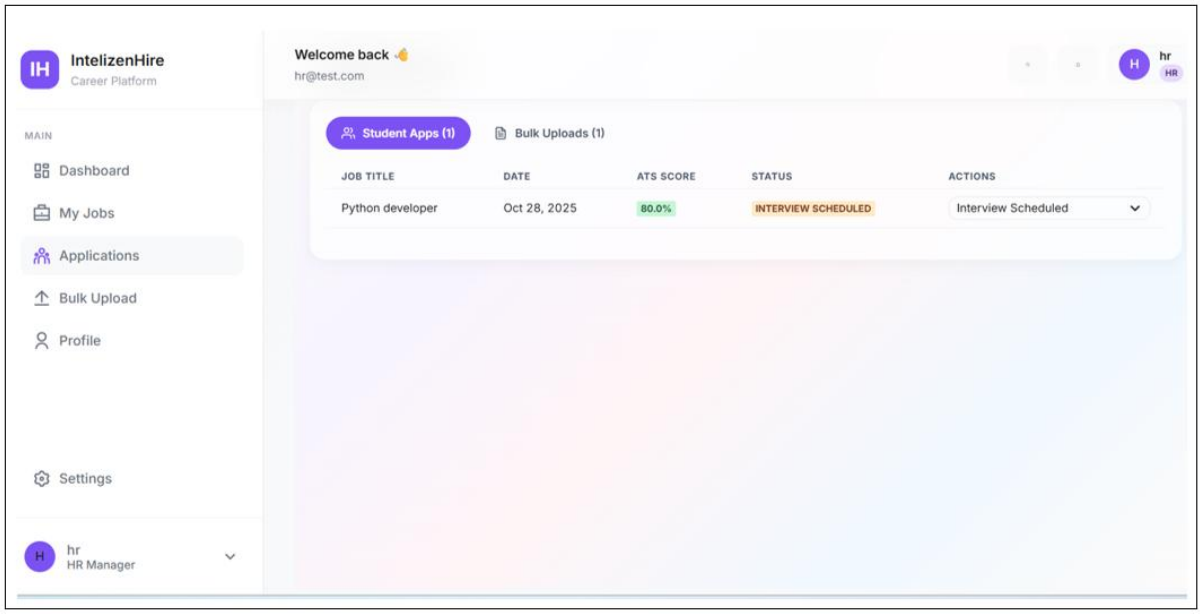


Figure 17: HR Application Manager

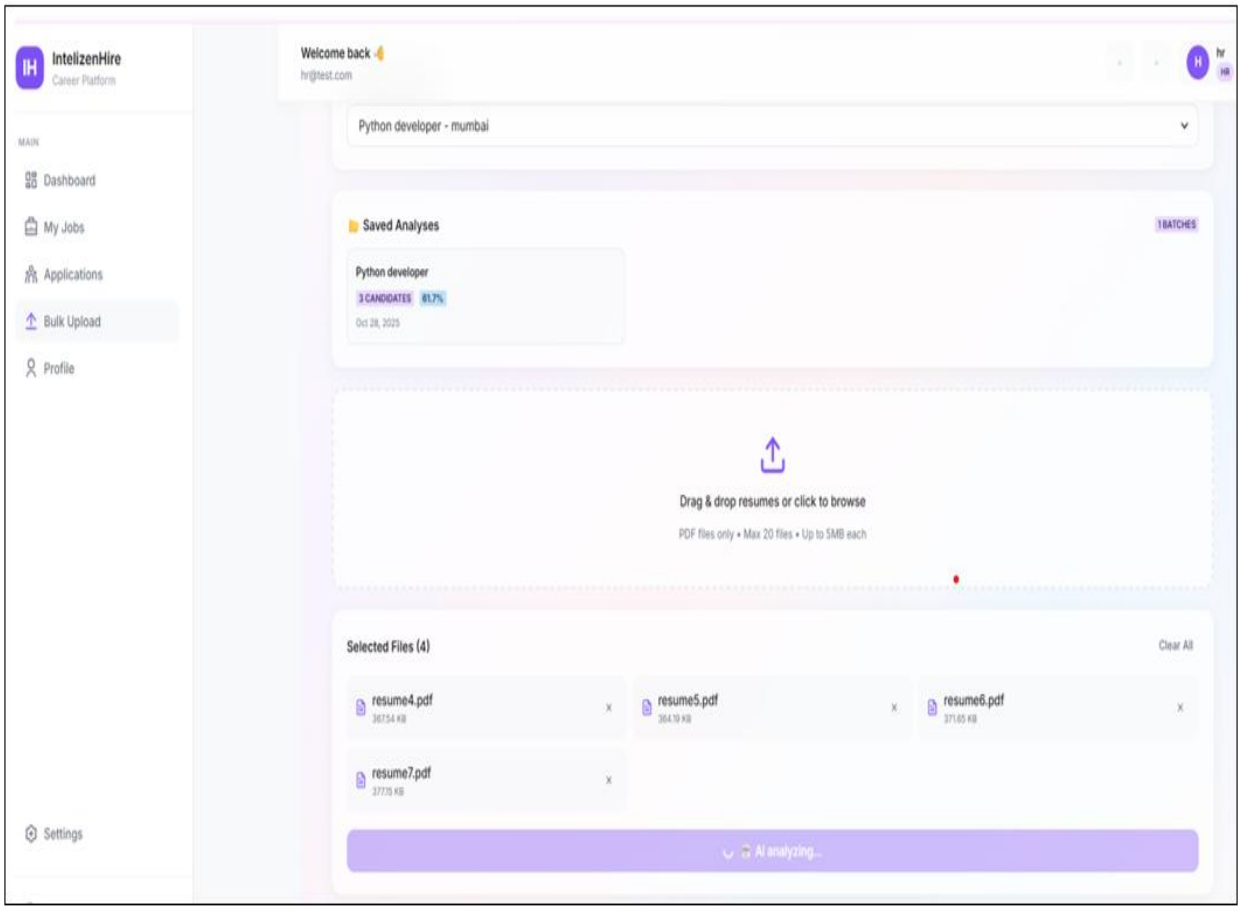


Figure 18: HR Bulk Resume Upload

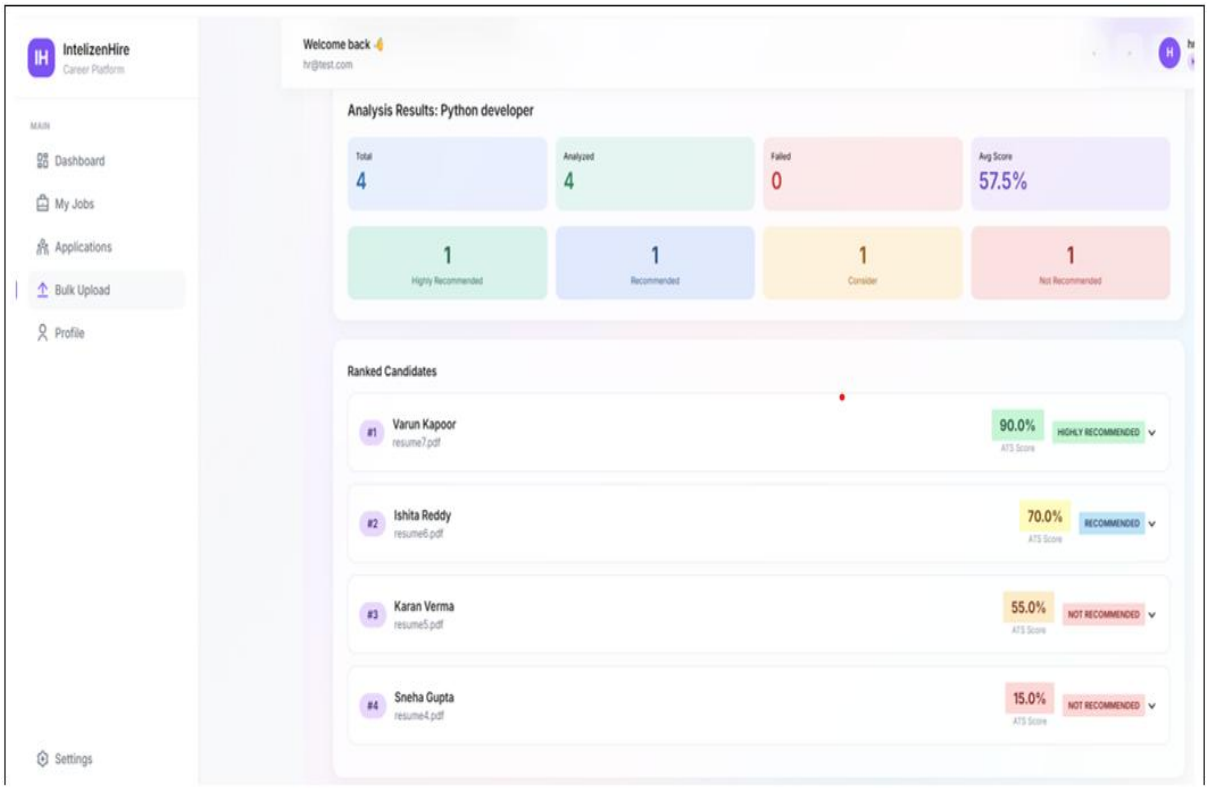


Figure 19: HR Candidate Ranking

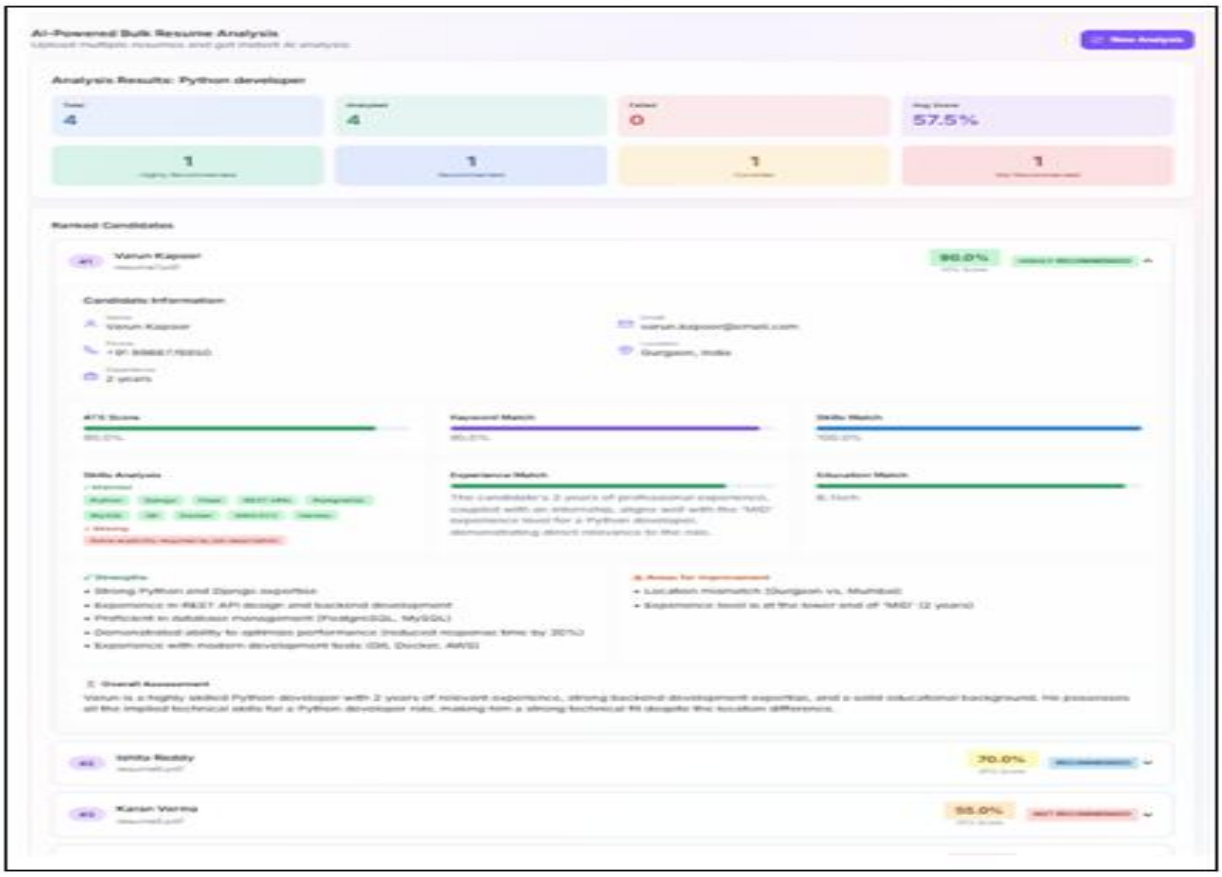


Figure 20: HR Candidate Ranking Analysis

Chapter 8

Conclusion and Future Work

8. Conclusion and Future Work

8.1 Conclusion

This work presented an NLP-Based Resume Analysis and Ranking System that addresses the limitations of traditional recruitment methods such as keyword-based filtering and manual screening. The system integrates NLP, machine learning and lightweight deep learning into a modular architecture supporting both student evaluation and recruiter decision-making.

A key strength is its hybrid evaluation approach, combining deterministic scoring with semantic understanding. Using TF-IDF, sentence-transformer embeddings, formatting checks, skill matching and education alignment, the system ensures interpretable and reproducible evaluation through a transparent weighted ATS scoring mechanism.

The skill gap detection module further enhances functionality by combining ontology-based expansion with zero-shot NLI to identify and quantify missing skills. This enables actionable feedback and supports student improvement, complemented by a recommendation layer that remains separate from the core pipeline.

For recruiters, the system provides a learning-based ranking framework integrating structured features and semantic similarity, with SHAP-based explainability offering feature-level insights. The architecture, built with FastAPI, FAISS and SQLite, ensures low latency, privacy and reproducibility, while separating deterministic and non-deterministic components for consistency.

Overall, the system improves recruitment efficiency, transparency and fairness by enabling automated evaluation, clear feedback and data-driven decisions. It bridges the gap between traditional ATS tools and modern AI-driven solutions through an interpretable and scalable framework.

8.2 Future Work

The proposed system establishes a strong foundation for intelligent and explainable resume analysis, with scope for further enhancement in adaptability, scalability and robustness. Future work can extend its capabilities to better support real-world, large-scale recruitment environments. Key areas include:

Adaptive Scoring Mechanisms: Replace fixed weights with dynamic, data-driven or learning-to-rank approaches to tailor evaluation based on job roles and industry needs.

Multi-Modal Candidate Evaluation: Incorporate additional inputs such as portfolios, coding assessments, certifications and interview performance for holistic evaluation.

Dynamic Skill Ontology Expansion: Continuously update skill ontologies using evolving knowledge bases and domain-specific skill graphs to capture emerging trends.

Scalability and Distributed Deployment: Transition to cloud or distributed architectures using microservices, distributed indexing and parallel processing to handle large-scale workloads efficiently.

References

[1] W. Li and X. Zhou, “Machine learning-based HR management information retrieval and classification algorithm,” *Scalable Comput.: Pract. Exp.*, vol. 25, no. 6, pp. 5431–5440, 2024.

[2] Sougandh, T. Giri, N. S. Reddy and M. Belwal, “Automated resume parsing: A natural language processing approach,” in *Proc. 7th Int. Conf. Comput. Syst. Inf. Technol. Sustainable Solutions (CSITSS)*, IEEE, 2023, pp. 1–6.

[3] M. Bocharova, E. Malakhov and V. Mezhuyev, “VacancySBERT: Representation of titles and skills for semantic similarity search,” *arXiv preprint* arXiv:2307.16638, 2023.

[4] M. Naudé, K. J. Adebayo and R. Nanda, “A machine learning approach to detecting fraudulent job types,” *AI Soc.*, vol. 38, no. 2, pp. 1013–1024, 2023.

[5] R. Vivek, “Enhancing diversity and reducing bias in recruitment through AI,” *Inf. Econ. Manag.*, vol. 2, no. 4, pp. 101–118, 2023.

[6] M. M. Tobroni and I. R. A. Hamid, “Secured recruitment management system with data retention approach,” *Appl. Inf. Technol. Comput. Sci.*, vol. 4, no. 2, pp. 230–247, 2023.

[7] A. Upadhye, “Illuminating HR insights: Unraveling patterns through data analytics and visualization,” *J.*, vol. 2811, p. 6201, 2023.

[8] L. Mostafa and S. Beshir, “Job candidate rank approach using machine learning techniques,” in *Proc. Int. Conf. Adv. Mach. Learn. Technol. Appl.*, Springer, Cham, Switzerland, 2021, pp. 225–233.

[9] P. Bhandari, S. Kumar and A. Singh, “Resume screening using hybrid deep learning model,” in *Proc. Int. Conf. Mach. Learn. Smart Educ. (ICMSLE)*, Springer, Singapore, 2024, pp. 89–98.

[10] J. Tan, “An automated hybrid resume NER based on rule-based and transformer approaches,” GitHub repository, 2024. [Online]. Available: https://github.com/academic. [Accessed: Mar. 29, 2025].

[11] Stanford Sentia Team, “The hybrid AI-human combo that improves hiring success by 53%,” Stanford Univ., Stanford, CA, USA, Tech. Rep., 2025.

[12] J. Tan, “CV recommendation model using latent Dirichlet allocation (LDA),” *arXiv preprint* arXiv:2404.01234, 2024.

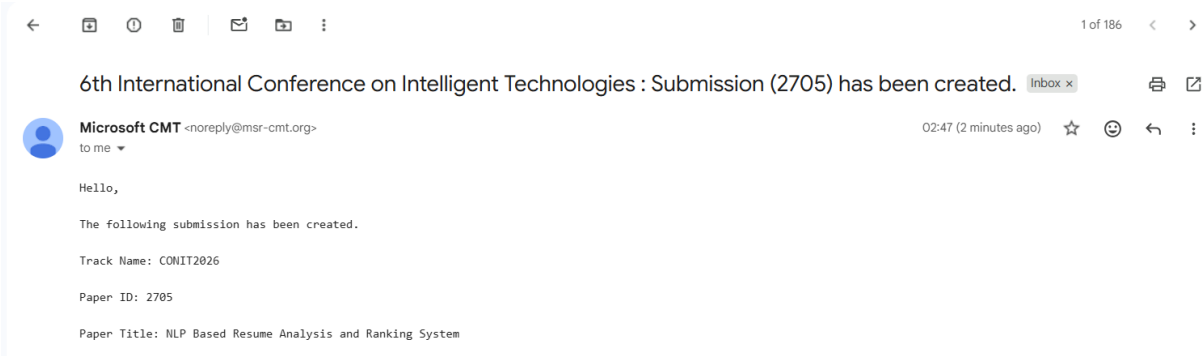
[13] Recruitics Res., “How AI-generated resumes impact the staffing industry,” Recruitics, Tech. Rep., 2025.

[14] B. Gaur, S. Sharma and P. K. Singh, “Semi-supervised deep learning based named entity recognition model to parse the education section of resumes,” *Neural Comput. Appl.*, vol. 33, no. 8, pp. 4211–4225, 2021.

[15] R. T. Fareed, M. A. Al-Sharafi and K. A. Rasheed, “Resume classification and ranking using KNN and cosine similarity,” in *Proc. Int. Conf. Comput. Inf. Sci.*, Springer, Cham, Switzerland, 2021, pp. 317–326

Publication

Paper entitled “NLP Based Resume Analysis and Ranking System” is submitted at “6th International Conference on Intelligent Technologies” by Prof. Vijaya Bharathi Jagan, Aabha Bhide, Aditi Gadhave, Niharika Bandekar, Sumeet Gupta.



1 of 186

6th International Conference on Intelligent Technologies : Submission (2705) has been created. Inbox ×

Microsoft CMT <noreply@msr-cmt.org>
to me

02:47 (2 minutes ago)

Hello,

The following submission has been created.

Track Name: CONIT2026

Paper ID: 2705

Paper Title: NLP Based Resume Analysis and Ranking System

NLP Based Resume Analysis and Ranking System

Aabha Bhide
Computer Science and Engineering(AI&ML)
A.P.Shah Institute of Technology
Thane, India
aabhabhide28@gmail.com

Vijaya Bharathi Jagan
Computer Science and Engineering(AI&ML)
A.P.Shah Institute of Technology
Thane, India
vbjagan@apsit.edu.in

Aditi Gadhave
Computer Science and Engineering(AI&ML)
A.P.Shah Institute of Technology
Thane, India
aditigadhave77@apsit.edu.in

Priyanka Patil
Computer Science and Engineering(AI&ML)
A.P.Shah Institute of Technology
Thane, India
pjpatil@apsit.edu.in

Niharika Bandekar
Computer Science and Engineering (AI&ML)
A.P.Shah Institute of Technology
Thane, India
niharikabandekar124@apsit.edu.in

Sumeet Gupta
Computer Science and Engineering (AI&ML)
A.P.Shah Institute of Technology
Thane, India
sumeetgupta69@apsit.edu.in

Abstract - Manual resume screening and keyword-based applicant tracking systems do not capture semantic relevance, provide actionable feedback and support interpretable candidate ranking. This paper presents an NLP-based resume analysis and ranking system designed for student self-assessment and recruiter decision support. The system computes a reproducible weighted Applicant Tracking System (ATS) score by combining Term Frequency–Inverse Document Frequency (TF-IDF) cosine similarity, sentence-transformer–based experience matching, rule-based formatting evaluation, explicit skill overlap and education alignment. A hybrid symbolic–semantic skill gap module extends job skills using an ontology and applies zero-shot natural language inference to estimate missing skill confidence. Recruiter-side candidate ranking adds structured candidate features with semantic similarity using a lightweight neural ranking model, supported by SHapley Additive exPlanations (SHAP)-based post-hoc explainability for top-k candidates. Implemented using FastAPI, Facebook AI Similarity Search (FAISS) and SQLite, the core ATS and ranking modules run locally with reproducible outcomes. The system supports SDG 4 (Quality Education) and SDG 8 (Decent Work and Economic Growth) through interpretable and reproducible hiring.

Keywords: Applicant Tracking System (ATS), Semantic similarity, Skill gap detection, Ontology-based skill mapping, Zero-shot natural language inference, SHapley Additive exPlanations (SHAP) explainability, FastAPI, Facebook AI Similarity Search (FAISS), SQLite

I. INTRODUCTION

In the modern job market, recruiters receive a mountain of resumes for every open role, making manual screening impractical, time consuming and ripe for human error. Conventional applicant tracking systems depend majorly on keyword matching, which fails to capture semantic context, often giving inaccurate final results. Solving this issue is important to increase hiring efficiency, reduce bias and make the process more transparent. Recruiters face challenges like increased cost and delay due to infective screening of candidates, while the candidates face issues like lack of clarity on criteria for evaluation and areas for improvement. Thus, an intelligent and explainable approach is essential.

To solve this problem, an NLP-based Resume Analysis and Ranking System handles resume screening using NLP, machine learning and Advanced deep learning. This system uses TF-IDF with cosine similarity for keyword matching, Sentence Transformer (MiniLM) embeddings with FAISS for semantic similarity, JobBERT for skill extraction and BART-based Natural Language Inference for skill-entailment analysis to produce explainable ATS scores and skill-gap insights. Implemented using a FastAPI backend, React with Chakra UI frontend and SQLite database, the system delivers a lightweight, efficient, secure and transparent platform for both recruiters and candidates. In this paper, design and implementation of an NLP-based resume analysis and ranking system is presented. It mentions the problem motivation, system architecture and methodology used for resume analysis and candidate ranking and also describes system functionality via architectural diagrams.

II. RELATED WORKS

Initial studies focused on using machine learning for candidate ranking based on resume features [8]. But these approaches relied on structured inputs and struggled with unstructured resume data. This gap was later mentioned in studies that introduced NLP based resume parsing techniques to extract structured entities such as skills and experience from unstructured documents [2]. To go beyond keyword-based matching, semantic similarity methods were introduced. Use of Sentence-BERT–based models improved candidate-job alignment by allowing contextual matching between the resumes and job descriptions [3].Later ethical and operational concerns were also studied. Explainable AI was emphasized to reduce bias and enhance transparency in hiring process [5], whereas secure recruitment systems focused on authentication and access control to protect sensitive data [6]. Newer studies shifted to candidate retrieval using ML-driven search and ranking models to deal with thousands of applications. [1]. While these studies address key pieces of automated hiring, they often function independently.

In summary, the current work contributes valuable solutions to individual stages of the hiring pipeline but lacks an integrated framework that combines semantic matching, skill validation, explainable ranking, analytics and security. The proposed system addresses this gap through an end-to-end, transparent and scalable recruitment architecture.

III. METHODOLOGY

This section of the paper presents the methodology implemented for the proposed NLP-Based Resume Analysis and Ranking System. The system supports two user roles, Student and HR through their respective web portals. It is a modular and layered architecture which allows automated resume screening, semantic job-candidate matching, explainable ranking and personalised feedback. The methodology section explains the general workflow, core system architecture and functioning of each module on both the sides, Student and HR. Figure 1 represents the system context of the resume analysis system, highlighting the date exchange taking place between students, recruiters and the core system, excluding the internal module-level details.

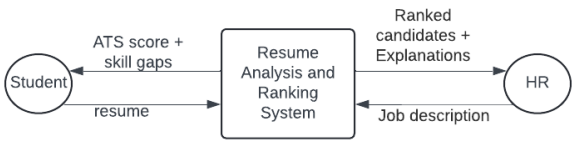


Fig. 1. System Context Diagram

A. System Overview

The proposed system automates candidate evaluation by using Natural Language Processing (NLP) and Machine Learning (ML) techniques to analyse resumes against job description. Students receive a detailed ATS report along with the feedback for improvement, while HR users receive ranked candidate backend along with a locally deployed AI engine, thus ensuring data privacy, real-time inference and scalability. Figure 2 represents the overview of the system.

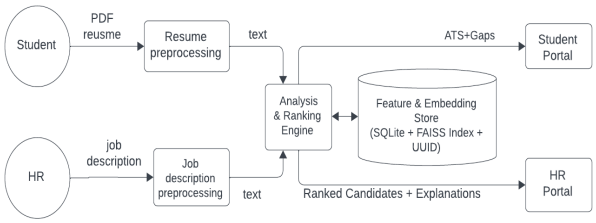


Fig. 2. Functional Architecture Diagram

B. Module-Specific Implementation

I) Student Modules:

Job Browsing & Application Module:

The Job Browsing and Application Module exposes a RESTful interface integrating a keyword-based filtering, optional semantic relevance enhancement and transactional application submission. Job discovery requests query a relational schema indexed on job status and text-search (tsvector) fields, by providing a paginated and filter aware result sets. Authenticated application submission is then processed through an idempotent write path enforced by a composite unique constraint, thus preventing duplicate applications. While submitting, an ATS compatibility score is computed synchronously, the application record is persisted within an ACID-compliant transaction and a Redis stream event is emitted enabling downstream analytics and monitoring. The module is designed to preserve transactional integrity and auditability.

Resume Ingestion and Parsing Module:

The module handles the acquisition, normalization and structured storage of resume data. PDF and DOCX files are accepted making use of PyPDF2 and python-docx. Text extracted is pre-processed including removal of extra whitespaces, converting to lowercase, normalizing Unicode characters (Normalization Form Compatibility Decomposition-NKFD) and eliminating special characters except . , ; : ! ? ( ) [ ] / @. Rule-based section segmentation using predefined regex patterns is used to identify the Education, Experience, Skills, Projects, Certifications, Languages and Interests sections while storing each section separately. Preliminary skill extraction is performed through keyword matching against a curated skill-words list. The original files and processed text are later stored in the SQLite table with ISO 8601-formatted timestamps.

Job-Match Analysis Module:

The module calculates the compatibility between a student's resume and a target job description by computing a weighted Applicant Tracking System (ATS) score. It evaluates five independent resume attributes namely keyword relevance, resume formatting quality, experience relevance, skill matching and educational alignment. Keyword relevance is given the highest weight of 30% to capture lexical correspondence between the resume and job description. Experience relevance is assigned 25%, measuring the semantic similarity between resume experience sections and job requirements. Resume formatting quality contributes 20% to ensure ATS-friendly structure and readability. Skill matching weighs 15% by evaluating explicit alignment with required job skills, while educational alignment holds 10% of the remaining weight to confirm degree-level eligibility. Each component score is later normalized and combined by making use of a fixed weighted aggregation scheme thus ensuring interpretability, consistency and deterministic evaluation without having to rely on external large language model APIs.

Let the normalized component scores be defined as follows:

K: Keyword relevance score

F: Resume formatting score

E: Experience semantic similarity score

S: Skill match score

D: Education match score

Keyword Relevance Score (TF-IDF Cosine Similarity)-

The keyword score is computed using TF-IDF vectorization followed by cosine similarity between the resume text $R$and the job keyword document J:

$$K = \cos(\mathbf{v}_R, \mathbf{v}_J)$$

where $\mathbf{v}_R$ and $\mathbf{v}_J$ denote TF-IDF vectors constructed over unigrams and bigrams.

Formatting Score (Heuristic Aggregation)-

Resume formatting quality is assessed using rule-based heuristics capturing structural compliance:

$$F = \frac{H + B + L}{3}$$

where,Hindicates the presence of required section headers, B denotes bullet-point density and L represents compliance with an optimal resume length range.

Experience Match Score (Semantic Similarity)-

The resume is split into overlapping text chunks $\{r_i\}$. Each chunk and the job description $J$are embedded into a shared vector space using a sentence-transformer model. The experience score is defined as the maximum cosine similarity:

$$E = max_i \cos(\mathbf{e}_{r_i}, \mathbf{e}_J)$$

This captures the strongest semantic alignment between any resume experience segment and the job requirements.

Skill Match Score-

Explicit skill overlap is computed as:

$$S = \frac{|S_r \cap S_j|}{|S_j|}$$

where S_r represents the set of skills extracted from the resume and $S_j$represents the required job skills.

Education Match Score-

Educational alignment is modelled as a binary score:

$$D = \begin{cases} 1, & \text{if a required degree keyword is present} \\ 0, & \text{otherwise} \end{cases}$$

Final ATS Score-

The overall ATS score is computed using fixed weights that sum to unity:

$$\text{ATS}_{score} = 0.30K + 0.20F + 0.25E + 0.15S + 0.10D$$

$$\text{ATS}_{\%} = \text{ATS}_{score} \times 100$$

Example:

Let the job description for a Junior AI Engineer be as follows:

Location: Mumbai (Hybrid)

Experience Level: Entry-Level (0–1 year)

Employment Type: Full-time

Required Skills: Python, Machine Learning, TensorFlow, Scikit-learn, OpenCV, SQL, Git, Problem Solving, Team Collaboration

Preferred Education: Bachelor of Engineering (B.E.) in Computer Science, Artificial Intelligence, or related field

Experience Expectation: Full-time, in-person internship or project experience in AI/ML

Let, the following resume be tested against the above mentioned job description:

Final-year B.E. in Computer Science & Engineering (AI & ML specialization) at APSIT, Mumbai University. Aspiring AI & ML enthusiast with practical experience from internships and academic projects, passionate about applying technology to solve real-world problems.

Skills**:** Python, Java, C, JavaScript, PHP, HTML, CSS, JS, PHP, MySQL, Scikit-learn, TensorFlow, Keras, CNN, Regression models, AWS (EC2, S3, Data Pipelines), ETL SQL, OpenCV, Pandas, NumPy, Matplotlib, Git, Jupyter Notebook,

Education: B.E. in Computer Science & Engineering (AI & ML), APSIT, Mumbai University (2021–Present, entering 7th semester).

Calculating the ATS Score for the above resume, against the given job description:

Keyword Relevance (K): TF-IDF cosine (resume, job_keywords) = 0.80

Formatting Score (F): (Header present + bullet density (0.85) + resume length (400–800)) / 3 = 0.90

Experience Relevance (E): max_i cos (MiniLM(chunk_i), MiniLM(job)) = 0.75

Skill Match (S): | {Python, ML, TensorFlow, …} ∩ required| / |required| = 0.75

Education Match (D): Degree keyword "B.E." found, D = 1.0

Final ATS Score:

ATS_{score} = (0.30·0.80+0.20·0.90+0.25·0.75+0.15·0.75+0.10·1.0) = 0.8

$ATS_{\%}$ = 0.8 x 100 = 80%

Explainability and Feedback Layer-

To ensure transparency, the module converts deterministic component scores (K, F, E, S, D) into qualitative feedback. A rule-based format checker flags sub-optimal bullet usage, missing section headers and length violations, whereas a zero-shot Natural Language Inference (NLI) classifier calculates the confidence scores for missing job-relevant skills. These calculations are translated into structured suggestions like "add measurable outcomes", "insert a dedicated Skills section" and is then displayed along with the ATS percentage, thus helping students to improve their resume before the ranking and recruiter-side evaluation is performed without changing the underlying scoring formulation.

Rule-Based and Semantic Diagnostics

Student-facing explainability problem is accomplished without exposing model-level attribution techniques. Instead, deterministic scoring components are read using rule-based heuristics and semantic diagnostics to produce human-readable feedback. Thus, clarity and usability is prioritized while still preserving the reproducibility of the underlying evaluation pipeline.

Feedback Synthesis and Presentation-

Instead of giving raw numerical scores or low-level model internals, the system presents structured qualitative feedback. This includes area for improvement (e.g.: missing project sections, unquantified achievements), actionable improvement suggestions (e.g., adding measurable outcomes or explicit skill listings) and skill deficiencies along with the confidence indicators.

Skill Gap Module:

The module assesses a student's resume to identify job-relevant skills that are present as well as missing with respect to a given role. The module first expands the set of required skills using a predefined skill ontology to account for hierarchical parent–child relationships. Explicit skill presence is then detected through case-insensitive matching between the resume text and the expanded skill set. Skills not detected in the resume are further evaluated using a zero-shot natural language inference classifier, which estimates the confidence that the resume does not semantically express each missing skill. This hybrid symbolic–semantic approach enables interpretable and training-free skill gap detection, producing a structured output consisting of matched skills, missing skills and confidence scores for each identified gap.

To provide actionable help apart from skill gap analysis, the system provides an optional course recommendation layer which is performed strictly after the skill gap inference and a mock interview model.

Course recommendation module: For every missing skill that exceeds a predefined threshold limit, the module queries an external large language module-based recommendation service to recover upskilling resources.

Mock Interview module: To improve the coverage and role specificity apart from the static interview questions, this module makes use of an external large language model for augmenting interview questions. For the given job roles and skill profiles, the system queries the external model to produce additional role-specific interview question bank, which is subsequently filtered and embedded with the use of same sentence-transformer model used before retrieval.

Both these optional modules are architecturally dissociated from the core functions of the system and does not change the core detection of the present skills, confidence score estimation or ATS score calculation or the similarity computation. Since both the modules make use of external API, they are non-deterministic and replaceable, thus ensuring reproducibility and auditability among the core analytical modules.

Application Tracking & Progress Monitoring Module:

The module collects student interactions and evaluation outputs and gives a combined view of resume usage and

application progress. It calculates summary statistics which includes resume and application counts, interview status transitions and average ATS performance. Application progress is summarized using a status-wise distribution derived from distinct application state transitions. All metrics are calculated by using deterministic aggregation functions over locally stored relational data, which ensures transparency, low-latency analytics and reproducibility.

II) HR facing module

Job Posting Management Module:

The module delivers a secure RESTful interface for establishing and managing job vacancies. A FastAPI router verifies incoming job data against a Pydantic model and implements role-based access control using JWT verification, restricting job creation to HR users. Validated postings are persisted using ACID-compliant database operations, with consistency upheld through schema-level constraints, including CHECK conditions enforce positive values and min_salary ≤ max_salary and categorized job status options. Each posting is linked with the user's HR profile to ensure their ownership and modification. An optional issuing flag enabling instant enablement of the opening triggers an event-based notification through Redis streams, allowing semantic indexing parts to refresh job encodings asynchronously. This design assures coherence, security and punctual accessibility of job data for suggestion and corresponding workflows.

Application & Interview Management Module:

The module administers the complete cycle of job entries using role-based APIs, a reliable finite-state device and exchange-safe retention, ensuring accuracy, accountability and actual-time system promptness.

Students send applications through POST /jobs/{job_id}/apply. This method prevents replicate requests by implementing a distinct composite condition on (student_profile_id, job_id) at the database tier. Upon confirmation, the device calculates the ATS score internal utilizing the resume–role aligning component and adds a new log in the Submissions table with a tart status of SUBMITTED. The process is created to provide fast response which is apt for reactive user experience.

Deterministic State Transition Model

Each administration moves through a finite-state machine (FSM) whose status is matching the router-level enumeration:

$$S = \{\text{SUBMITTED}, \text{SHORTLISTED}, \text{INTERVIEW_SCHEDULED}, \text{INTERVIEWED}, \text{OFFERED}, \text{REJECTED}, \text{WITHDRAWN}\}$$

and action set

A= {submit, shortlist, schedule_interview, record_interview, offer, reject, withdraw}

OFFERED, REJECTED and WITHDRAWN are the final pointers; no further changes allowed.

State changes are done through PATCH /applications/{application_id}/status, open solely to HR-role users. A partial transition operation,

$$\delta: S \times A \rightarrow S$$

is enforced at the service layer such that only workflow-legal transitions are accepted. Invalid transitions (e.g., REJECTED → OFFERED) are rejected, guaranteeing deterministic and valid application progression.

Interview Scheduling Semantics

Converting an entry to INTERVIEW_SCHEDULED sets the state only; date, time and mode are saved subsequently by means of a follow-up POST/applications/{id}/interview-details API point, preserving the central flow separated.

Access Control and Data Isolation-

Row-level access control guarantees HR users can act only on entries of their job listings, implemented by the requirement:

$$\text{application.job_description.hr_profile_id} = \text{current_user.hr_profile.id}$$

This assures recruiter ownership and restricts inter-vacancy conflict.

Audit Logging and Event Propagation-

All the state changes are fixatedly audit-recorded, recording the last state, current status, operating user ID and time stamp. Each successful mutation emits a structured event (application_status_changed) to a Redis Stream consumed via user groups for resilience.

HR Dashboard and Progress Visualization-

A paged GET /applications API enables recruiter dashboards, facilitating sorting by ATS score or entry submission time. The application progress signal displayed in the HR shell presents a phase-based proxy coordinated with FSM phases, rather than an absolute gross total metric.

Bulk Resume Upload Module:

The module allows HR to process several candidates resumes in one action. The app stores ZIP archives having several PDF or DOCX files, extracts text data using PyPDF2 and python-docx and employs the same reliable ATS scoring workflow used for each and every resume analysis. To avert duplication processing, an encrypted hash is calculated for each & every resume and measured against current records to spot redundant entries.

The retrieved text and related metadata are continuously stored beside the original data, allowing optimized recycle by backend analysis and ranking functions. In initial testing with batch loads, the system displayed a practically observed average computing time of 4.5s per resume, refinement for big-scale databases is saved for later work.

Candidate Ranking & Explainability Module:

After the resume upload and ATS-based analysis, shortlisted candidates are graded using a composite learning-based model enhanced with retrospective understanding. The goal of this function is to make a stable, clear ranked list of candidates/students while still keeping recruiters trust and decision clarity

Each candidate is encoded as a fixed-length numeric feature vector:

$$x_c = \begin{bmatrix} ATS_c, \\ YearsExp_{c,} \\ SkillMatch_{c,} \\ EduLevel_c, \\ LocationMatch_{c,} \\ SalaryFit_c \end{bmatrix}$$

where ATS_c denotes the deterministic resume–job compatibility score, $YearsExp_c$ represents years of professional experience, $SkillMatch_c \in [0,1]$ is the proportion of required skills matched, $EduLevel_c$ is an ordinal encoding of education level, $LocationMatch_c \in \{0,1\}$ indicates geographic compatibility and $SalaryFit_c$ models the alignment between the candidate's expected salary s_c and the job budget b. $SalaryFit_c$ is clipped to [0,1] to ensure bounded values. All features are standardized using precomputed mean and variance parameters to ensure numerical stability.

Candidate relevance is estimated using a lightweight feed-forward neural network (RankNet) function. The network outputs a scalar relevance score for each candidate. Model parameters are pre-trained and loaded in evaluation mode at inference time, ensuring deterministic and reproducible ranking behaviour.

To incorporate semantic alignment beyond structured attributes, cosine similarity is computed between the job description and candidate resume text using sentence-level embeddings. FAISS is employed for efficient similarity computation. The final ranking score is obtained via a weighted ensemble:

$$Score_c = 0.7 \cdot f(x_c) + 0.3 \cdot s_c^{sem}$$

where s_c^{sem} denotes the semantic similarity score. Candidates are ranked in descending order of this combined score.

Explainability via SHAP-

SHAP (SHapley Additive exPlanations) is applied exclusively within the HR Candidate Ranking module to provide transparent, recruiter-facing explanations of the final relevance score. For each top-ranked candidate, SHAP DeepExplainer computes per-feature attributions that quantify the influence of factors such as ATS score, years of experience, skill-match ratio, education level and related attributes on the ranking outcome. Even though SHAP credit values are formed internally in the HR Candidate Ranking function, the current HR portal doesn't touch these outputs graphically. The module outputs an ordered set of candidates, where each entry includes an overall relevance metric, a recommendation category (Highly Recommended / Recommended / Not Recommended) and human-readable, feature-level interpretability details. All processing is executed locally with no dependency on external APIs, which supports low-latency performance, consistent results and strong data protection.

IV. RESULTS

System Interface and Module Results

a) Student Portal

Resume Job-Match Insights: When each Resume is uploaded, it is immediately followed by a 0-100% ATS Score that is segregated into five colour-coded bars: keyword, skill, format, education and experience. Students get suggestions like "Missing Python skill detected" or "Experience contributed positively", helping students in a self-improving support system that guides and coaches them for revision without external coaching. The score is cloned using the same TF_IDF, rule-based pipeline, MiniLM, ensuring full inspection.

Skill-Gap Discovery: The matched vs. missing skills presented by a zero-shot entailment classifier as clickable chips are marked up with confidence badges; a more extensive ontology layer (e.g., React → JavaScript) prevents synonyms' blind side. The result prioritises gap list links to a course catalogue (present locally but queried via an external API), while maintaining training-free, cloud-free skill detection, it transforms the diagnostic result into an actionable learning path.

Resume Analysis: The local hybrid pipeline of 5.B.III instantly renders the student side analysis screen by rendering the deterministic ATS Score (0-10%), it then decomposes the score into colour-coded "missing /matched skills" chips that directly reflect the ontology-expanded, zero-shot NLI results from the Skill Gap Module. within the same MiniLM + SQLite stack, every improvement bullet and confidence badge is generated solely from the rule-based and semantic diagnostics, ensuring the whole feedback experience remains cloud-free, reproducible and fully aligned with the methodological accuracy.

Semantic Job Recommendations: The "Jobs You Might Like" recommends scrollable job cards, automatically matching the student's profile, omitting explicit search items. This uses cosine similarity, which helps in context-aware discovery and encourages exploratory browsing. Local MiniLM encoder and FAISS flat index are used to compute all similarities.

Mock-Interview Generation: A local FAISS index primarily retrieves interview questions over a curated question bank by combining the resume and preferred job types as a query. An external LLM, i.e. Large Language Model API, can optionally generate additional role-aligned questions to enhance coverage. The same sentence-Transformer filters and embeds the newly generated questions. Since the augmentation layer is built separately, not as tightly connected, the core semantic retrieval pipeline remains reproducible, methodologically consistent and auditable.

Activity Timeline & Self-Monitoring: A chronological list of events records every status change so nothing is lost (submitted → shortlisted → interviewed →offer→ Accepted) to local ISO-8601 timestamps. The progress bar converts discrete states into perceived advancement, making the user feel the progress and stay motivated, which improves engagement without predictive modelling or cloud telemetry.

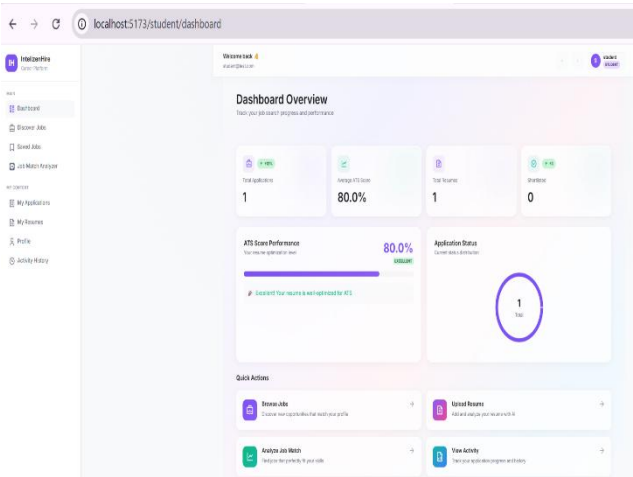


Fig. 3. Student Dashboard

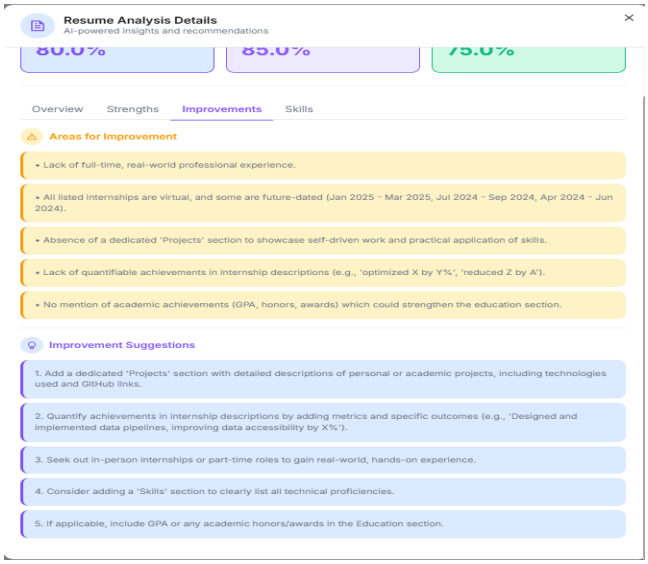


Fig. 4. Descriptive resume analysis

.b) HR Portal

Bulk-Upload & Analysis: Uploading multiple PDF resumes by dragging Triggers the local PyPDF2 + MiniLM pipeline, which yields instant ATS score bars and a rule-based ranked list produced with the help of the RankNet + SHAP ensemble; the local relevance score helps to obtain the "Not Recommended" badge with no external ranking service consulted.

Application Pipeline: The finite-state-machine transition triggered by the PATCH endpoint is reflected by the 'INTERVIEW SCHEDULED' chip; the SQLite Applications table stores timestamp and ATS score locally, so no cloud logging is needed, which ensures full auditability and each transition is permanently logged and propagated via Redis streams.

Job Posting Management: The 'Create New Job' form gets filled automatically with skills, degrees and company names using the JobBERT NER extractor applied locally to the free-text job description. A Redis event is activated with the 'Publish immediately' toggle, which triggers a local FAISS job index rebuild, keeping synchronised and cloud-free semantic search and recommendation feeds.

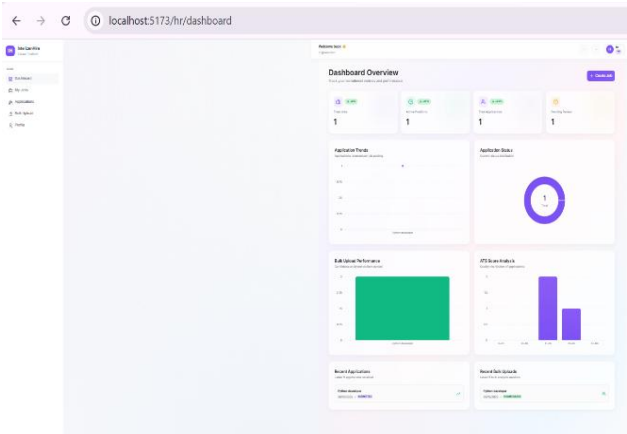


Fig. 5. HR Dashboard

V. CONCLUSION

This paper presents an NLP-based Resume Analysis and Ranking System which goes beyond the traditional keyword matching approach used for automating recruitment. The system provides accurate and transparent candidate–job matching by integrating semantic similarity modelling, skill gap analysis and interpretable ATS scoring within a modular architecture. Further inclusion of dedicated dashboards for both, recruiters and candidates enhance usability and decision-making. A major limitation of this system is its dependency on textual resume data and predefined scoring weights, which may need to be personalized for various hiring contexts. Future work focuses on integrating additional data sources like assessments, project portfolios and interview feedback with adaptive scoring mechanisms and bias-aware evaluation. The system can also be expanded further as an intelligent interview recommendation and analytics modules, as well as cloud-based deployment, which could support large-scale and real-time recruitment workflows.

VI. REFERENCES

[1] Wen Li and XiuKao Zhou, Machine Learning-Based HR Management Information Retrieval and Classification Algorithm, Scalable Computing: Practice and Experience, Vol. 25, No. 6, pp. 5431-5440, 2024.

[2] Sougandh, Thatavarthi Giri, Nithish Sagar Reddy and Meena Belwal, Automated Resume Parsing: A Natural Language Processing Approach, In 2023 7th International Conference on Computation System and Information Technology for Sustainable Solutions (CSITSS), IEEE, pp. 1-6, 2023.

[3] Maiia Bocharova, Eugene Malakhov and Vitaliy Mezhuyev, VacancySBERT: Representation of Titles and Skills for Semantic Similarity Search, arXiv preprint arXiv:2307.16638, 2023.

[4] Marcel Naudé, Kolawole John Adebayo and Rohan Nanda, A Machine Learning Approach to Detecting Fraudulent Job Types, AI & Society, Vol. 38, No. 2, pp. 1013-1024, 2023.

[5] Ramakrishnan Vivek, Enhancing Diversity and Reducing Bias in Recruitment through AI, Informatics. Economics. Management, Vol. 2, No. 4, pp. 0101-0118, 2023.

[6] Mashitah Mohd Tobroni and Isredza Rahmi A. Hamid, Secured Recruitment Management System with Data Retention Approach, Applied Information Technology and Computer Science, Vol. 4, No. 2, pp. 230-247, 2023.

[7] Akshata Upadhye, Illuminating HR Insights: Unraveling Patterns through Data Analytics and Visualization, Journal ID 2811, pp. 6201, 2023.

[8] Lamiaa Mostafa and Sara Beshir, Job Candidate Rank Approach Using Machine Learning Techniques, In International Conference on Advanced Machine Learning Technologies and Applications, Springer, Cham, pp. 225-233, 2021.

[9] P. Bhandari, S. Kumar and A. Singh, Resume Screening Using Hybrid Deep Learning Model, in Proc. Int. Conf. on Machine Learning and Smart Education (ICMSLE), Springer, Singapore, pp. 89-98, 2024.

[10] J. Tan, An Automated Hybrid Resume NER Based on Rule-Based and Transformer Approaches, GitHub Academic, 2024.

Acknowledgements

We take this opportunity to express our sincere thanks to our guide, Prof. Vijaya Bharathi Jagan, for providing the technical guidelines and suggestions regarding the line of work. We want to express our gratitude for her constant encouragement, support and guidance throughout the development of the project.

We thank our Head of Department, Dr. Jaya Gupta, for being encouraging throughout the course and for her guidance.

We express our gratitude towards the BE Project Coordinator, Prof. Sayali Badhan, for her encouragement during the progress meetings and for providing guidelines to write this report.

We also thank the entire staff of APSIT for their invaluable help rendered during the course of this work. We wish to express our deep gratitude towards all our colleagues of APSIT for their encouragement.

Aabha Bhide
(22106093)

Aditi Gadhave
(22106079)

Niharika Bandekar
(22106136)

Sumeet Gupta
(22106071)